

Visualisation de données avec R

Bioinformatique I (BIF-7900)

Antoine Bodein

15 octobre 2026

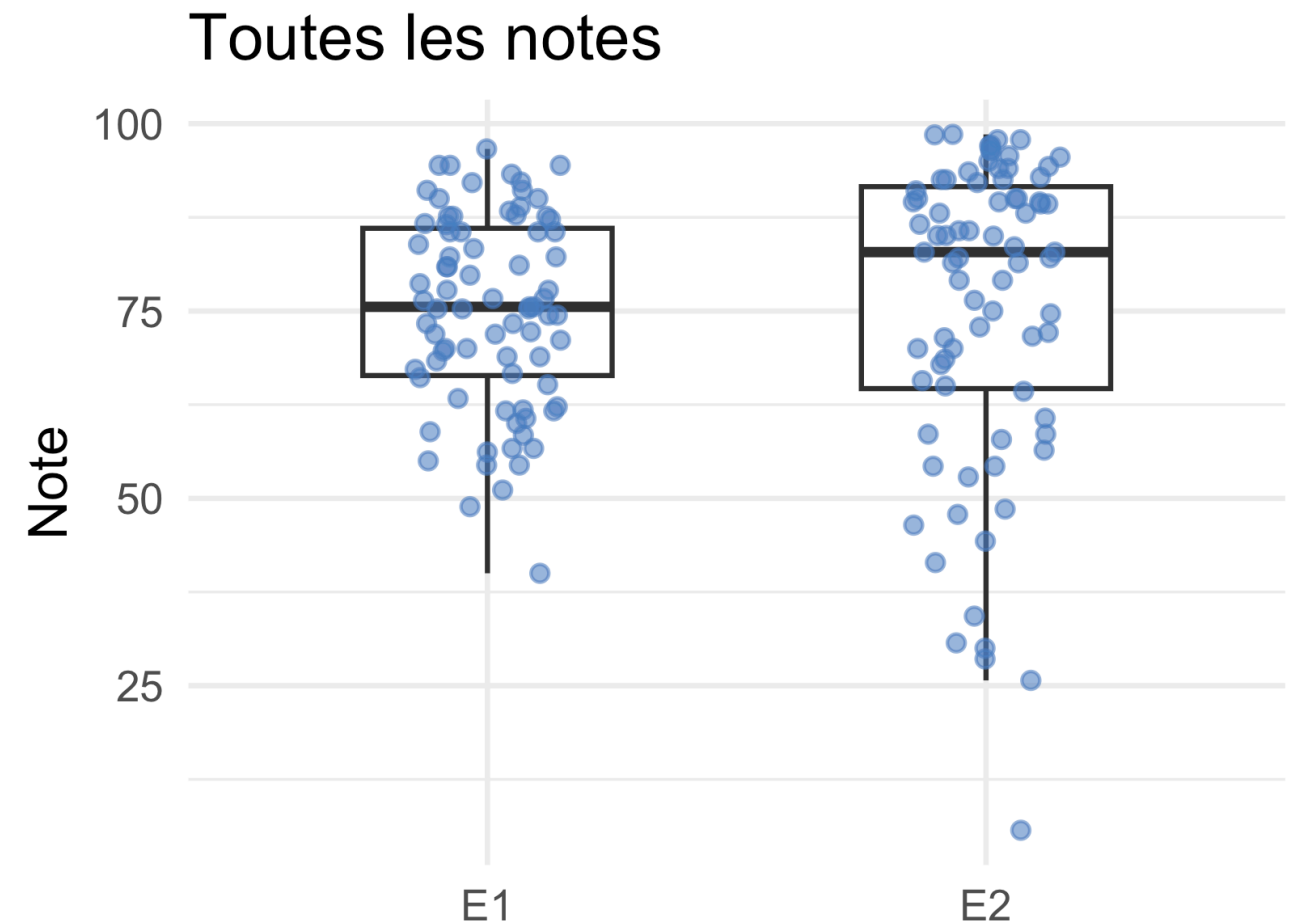
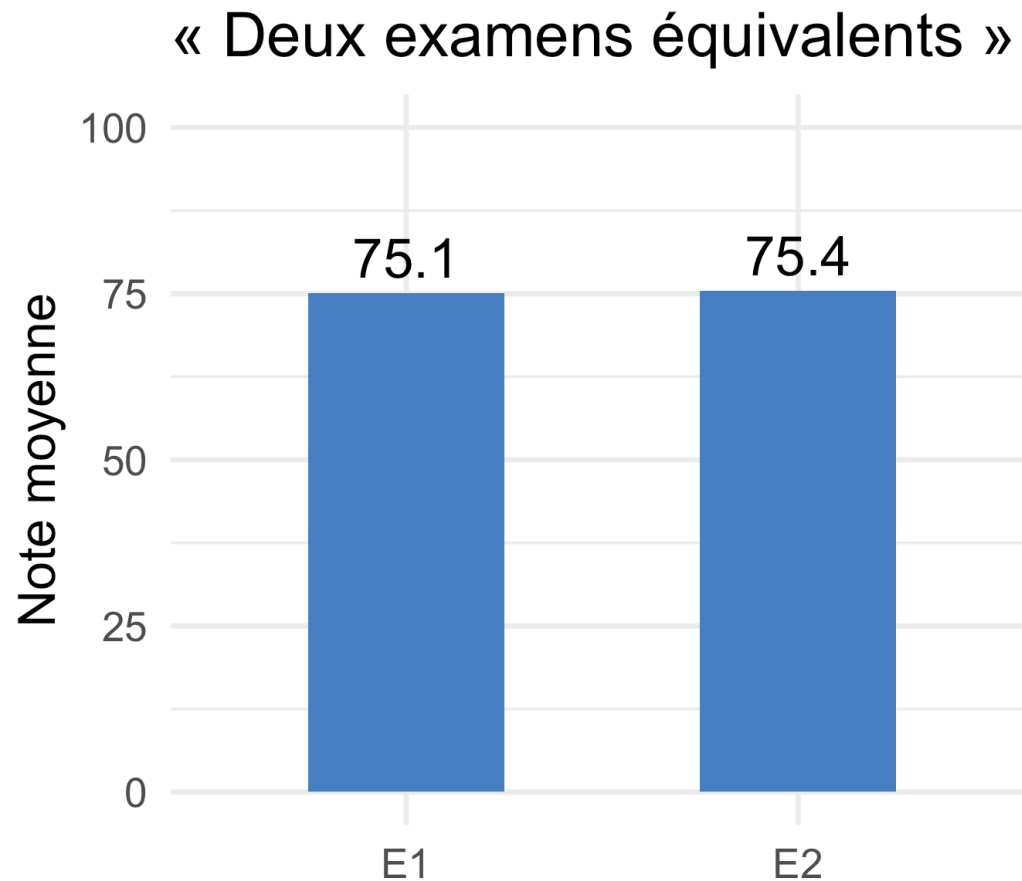


| 1

INTRODUCTION

Objectifs

Deux examens du cours, 79 étudiants (`grades.tsv`, module 6). Les deux moyennes :



Même moyenne, deux examens différents : E2 a une médiane plus haute, mais une dizaine d'échecs. Le barplot n'a rien caché par erreur : il **résume**, et le résumé était le mauvais choix.

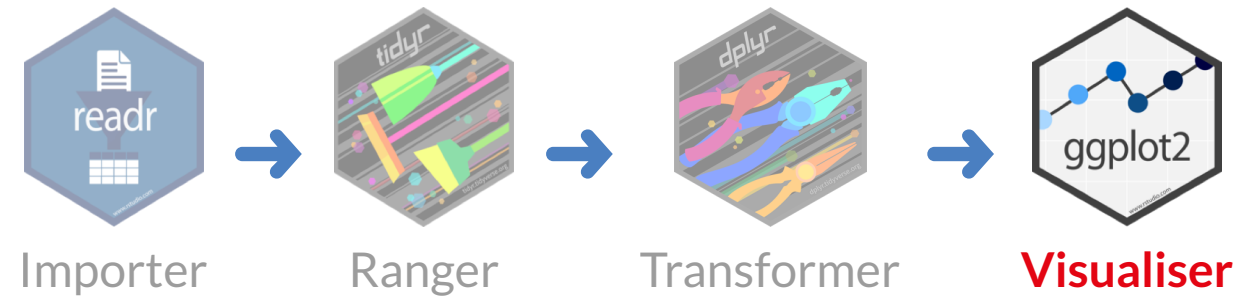
Objectifs

1. Faire les graphiques usuels avec **R de base** (`plot()`, `hist()`, `boxplot()`)
2. Comprendre la **grammaire** de `ggplot2` : données, *aesthetics*, *geoms*, couches
3. Ajuster un graphique : statistiques, positions, **échelles**, facettes, thèmes
4. **Préparer** les données pour `ggplot2` : format long, ordre des catégories, groupes
5. Construire les graphiques de la **bioinformatique** (volcano plot, heatmap) et **choisir** le bon graphique

Plan

1. Introduction
2. Les graphiques de R de base
3. La grammaire de **ggplot2** → **TP1**
4. Ajuster un graphique
5. Préparer les données → **TP2**
6. Graphiques de la bioinformatique et bonnes pratiques → **TP3**
7. Activité : auditer un script **ggplot2** généré par IA

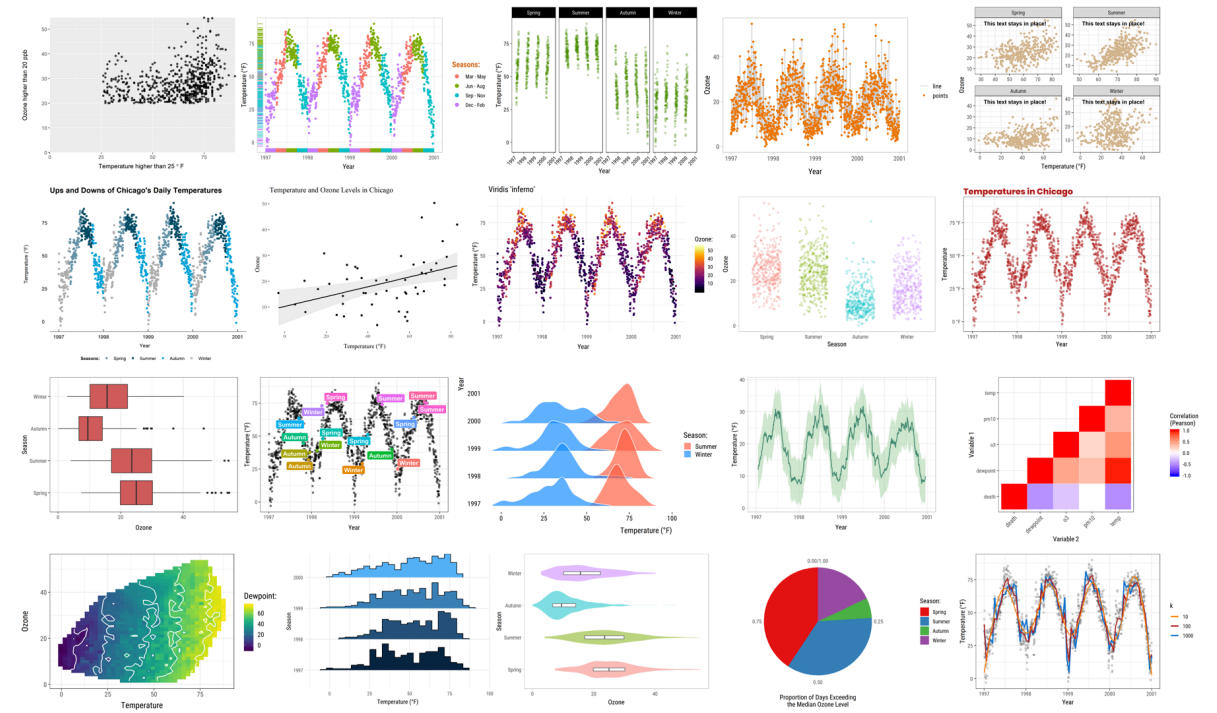
Où en est-on ?



- **Module 6** : importer (**readr**), ranger (**tidyr**), transformer (**dplyr**)
- **Aujourd'hui : visualiser.** On réutilise tout : un graphique **ggplot2** part d'un tableau **rangé**, souvent après un **pivot_longer()** ou un **summarise()**
- **Module suivant** : modéliser (statistiques), pour dire si ce qu'on voit sur le graphique dépasse le hasard

Pourquoi des graphiques en lignes de code ?

- **Reproductible** : le script refait le graphique à l'identique, sur les nouvelles données de la semaine prochaine
- **Modifiable** : changer une couleur, un seuil, un ordre, en une ligne
- **Partageable** : le code accompagne la figure de l'article
- Des milliers de packages spécialisés (génomique, réseaux, cartes)



Quelques graphiques **ggplot2** (C. Scherer, *A ggplot2 tutorial for beautiful plotting in R*)

Un graphique est souvent **la première vérification** d'une analyse : il montre les valeurs aberrantes, les **NA**, les groupes mal codés, les distributions bizarres, que les moyennes et les tests cachent. Un assistant d'IA écrit le code d'un graphique en une seconde ; **lire** le graphique, et voir ce qu'il ne montre pas, reste votre travail.

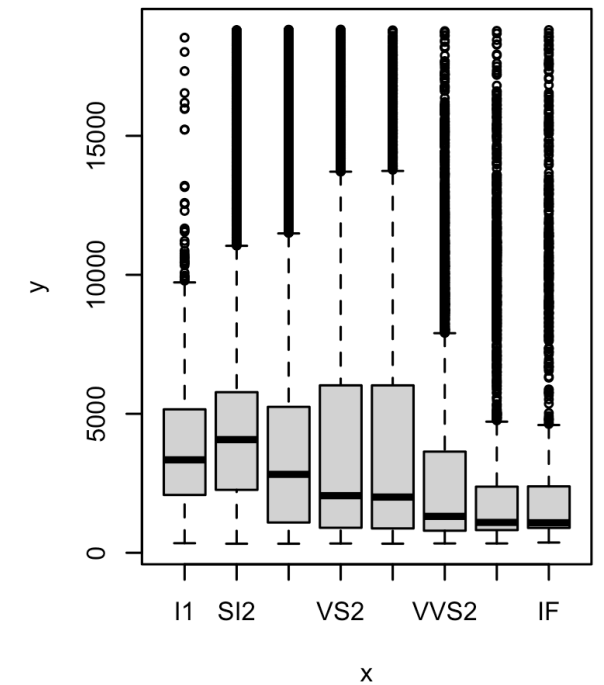
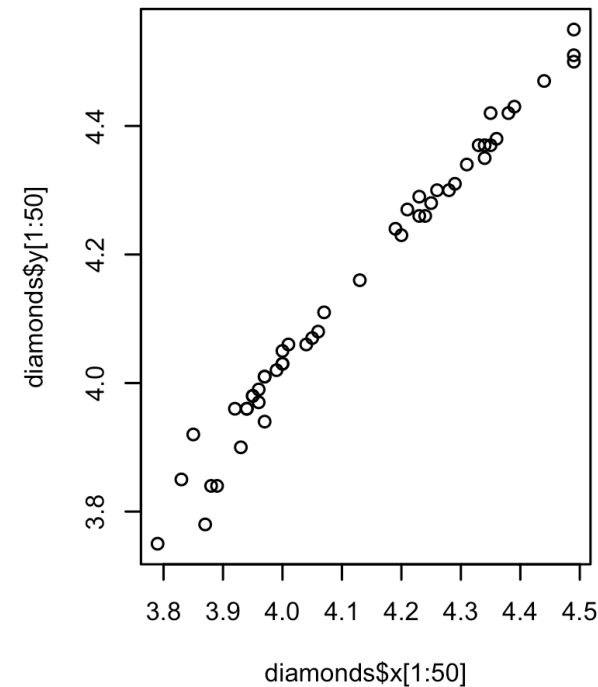
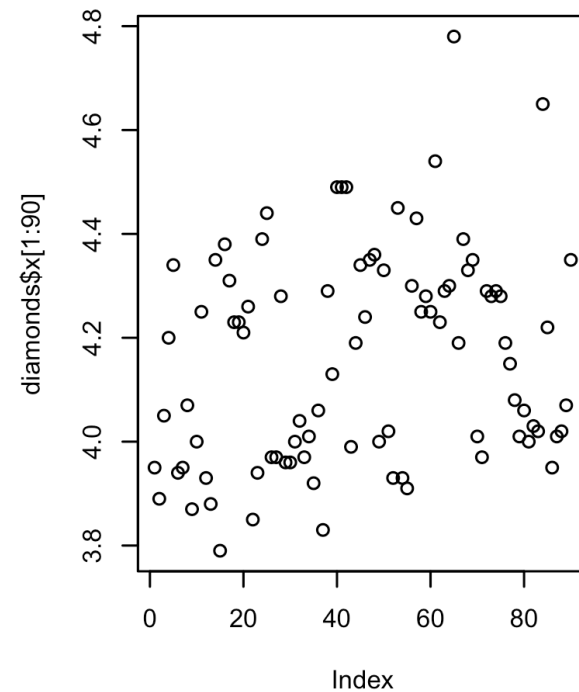
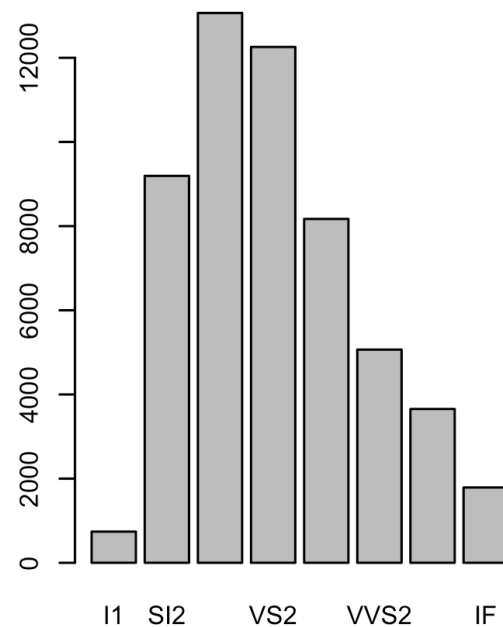
| 2

LES GRAPHIQUES DE R DE BASE

La fonction `plot()`

`plot()` choisit le graphique selon le **type** de ce qu'on lui donne :

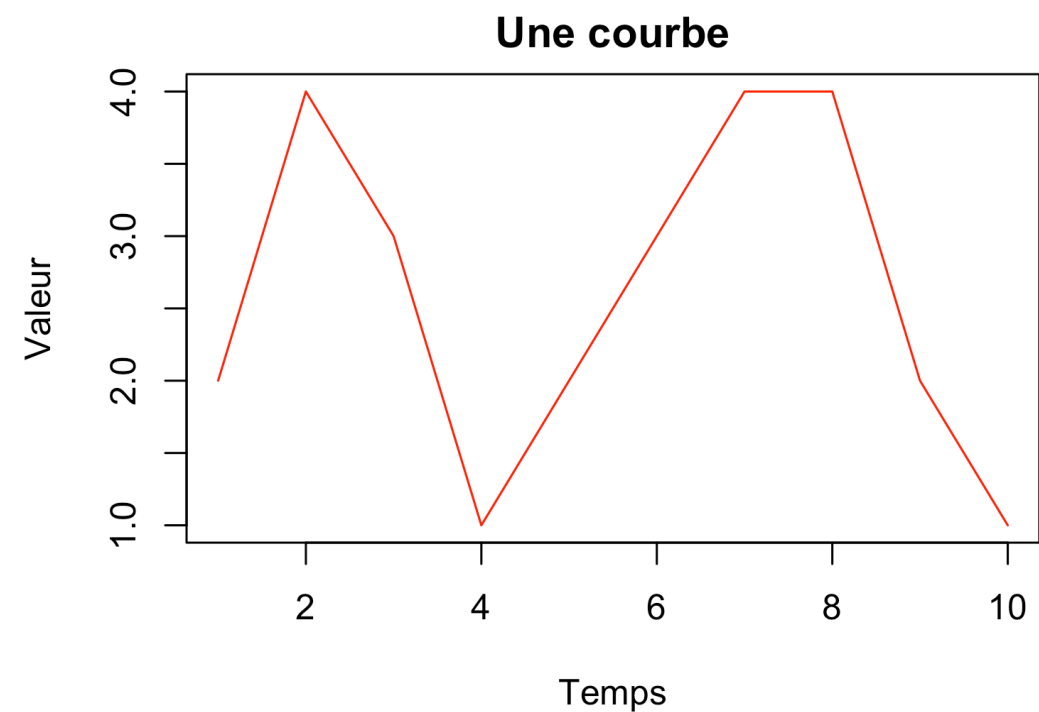
```
par(mfrow = c(1, 4))           # 4 graphiques côte à côte
plot(diamonds$clarity)          # un facteur : barplot des effectifs
plot(diamonds$x[1:90])          # un vecteur numérique : valeur selon la position
plot(diamonds$x[1:50], diamonds$y[1:50]) # deux numériques : nuage de points
plot(diamonds$clarity, diamonds$price) # facteur + numérique : boxplots
```



`diamonds` (inclus dans `ggplot2`) : prix, poids (`carat`), taille, couleur et pureté (`clarity`) de 53 940 diamants.

Les arguments de `plot()`

```
x <- 1:10
y <- c(2, 4, 3, 1, 2, 3, 4, 4, 2, 1)
plot(x = x, y = y, type = "l", col = "red",
     main = "Une courbe", xlab = "Temps",
     ylab = "Valeur")
```



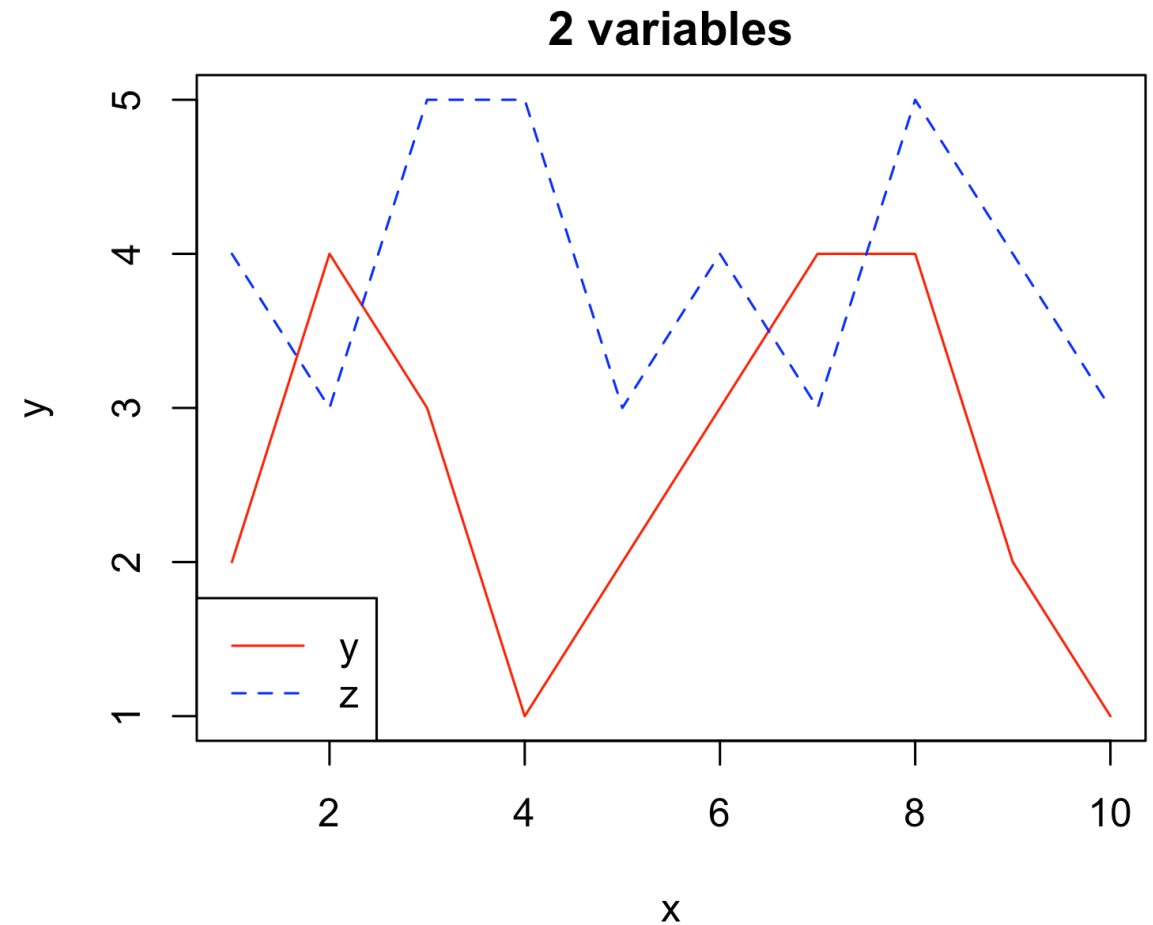
Argument	Rôle
<code>x, y</code>	les valeurs ; ou la formule <code>plot(y ~ x)</code>
<code>type</code>	"p" points, "l" lignes, "b" les deux
<code>main, xlab, ylab</code>	titre, titres des axes
<code>xlim, ylim</code>	limites des axes : <code>xlim = c(1, 3)</code>
<code>col</code>	couleur
<code>pch, lty</code>	forme des points, type de ligne

Toutes les options : `help(plot)`, `help(par)`

Ajouter des couches : **lines()**, **points()**, **legend()**

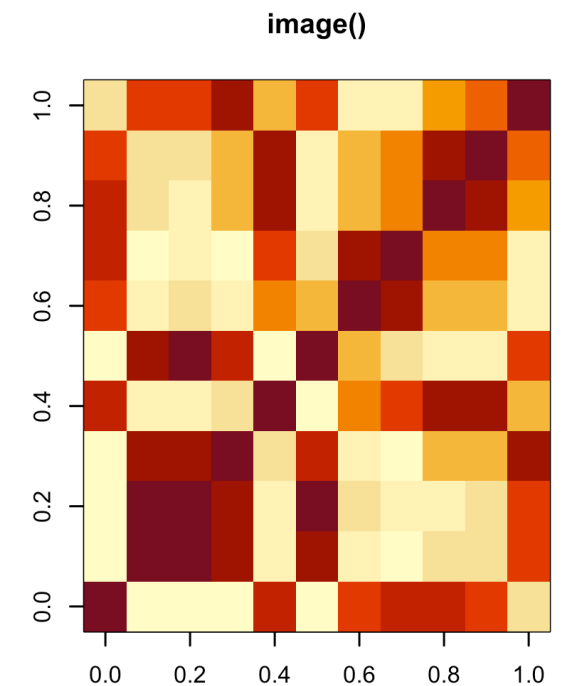
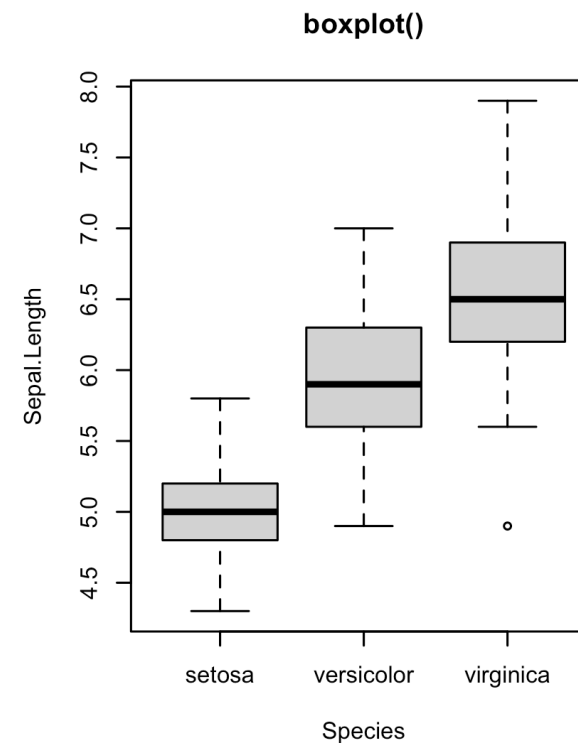
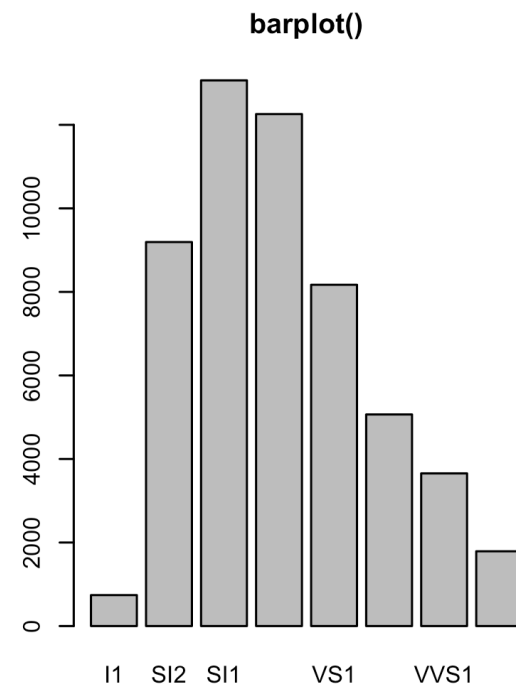
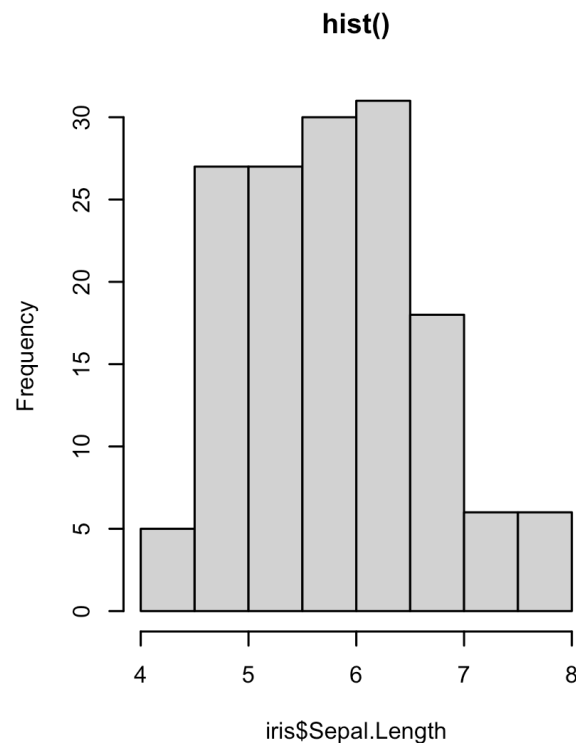
```
z <- c(4, 3, 5, 5, 3, 4, 3, 5, 4, 3)
plot(x, y, type = "l", col = "red",
     ylim = c(1, 5), main = "2 variables")
lines(x, z, col = "blue", lty = 2)
legend("bottomleft", legend = c("y", "z"),
     col = c("red", "blue"), lty = c(1, 2))
```

- **plot()** ouvre un graphique
- **lines()**, **points()**, **abline()**, **text()**, **legend()** ajoutent sur le graphique ouvert
- Les limites sont fixées par le premier **plot()** : d'où **ylim**



Histogramme, barplot, boxplot, heatmap

```
par(mfrow = c(1, 4))  
hist(iris$Sepal.Length, main = "hist()") # distribution d'un numérique  
barplot(table(diamonds$clarity), main = "barplot()") # hauteurs déjà calculées (effectifs)  
boxplot(Sepal.Length ~ Species, data = iris, main = "boxplot()")  
image(cor(mtcars), main = "image()") # une matrice en couleurs
```



iris : 4 mesures de fleurs (mm) pour 150 iris de 3 espèces (Fisher, 1936). **mtcars** : 11 caractéristiques de 32 voitures.

Sauvegarder un graphique

```
# 1. ouvrir un fichier graphique : png(), jpeg(), tiff(), pdf(), svg()
png("figures/mon_graphique.png", width = 1600, height = 1200, res = 200)

# 2. faire le graphique (il ne s'affiche pas : il s'écrit dans le fichier)
boxplot(Sepal.Length ~ Species, data = iris)

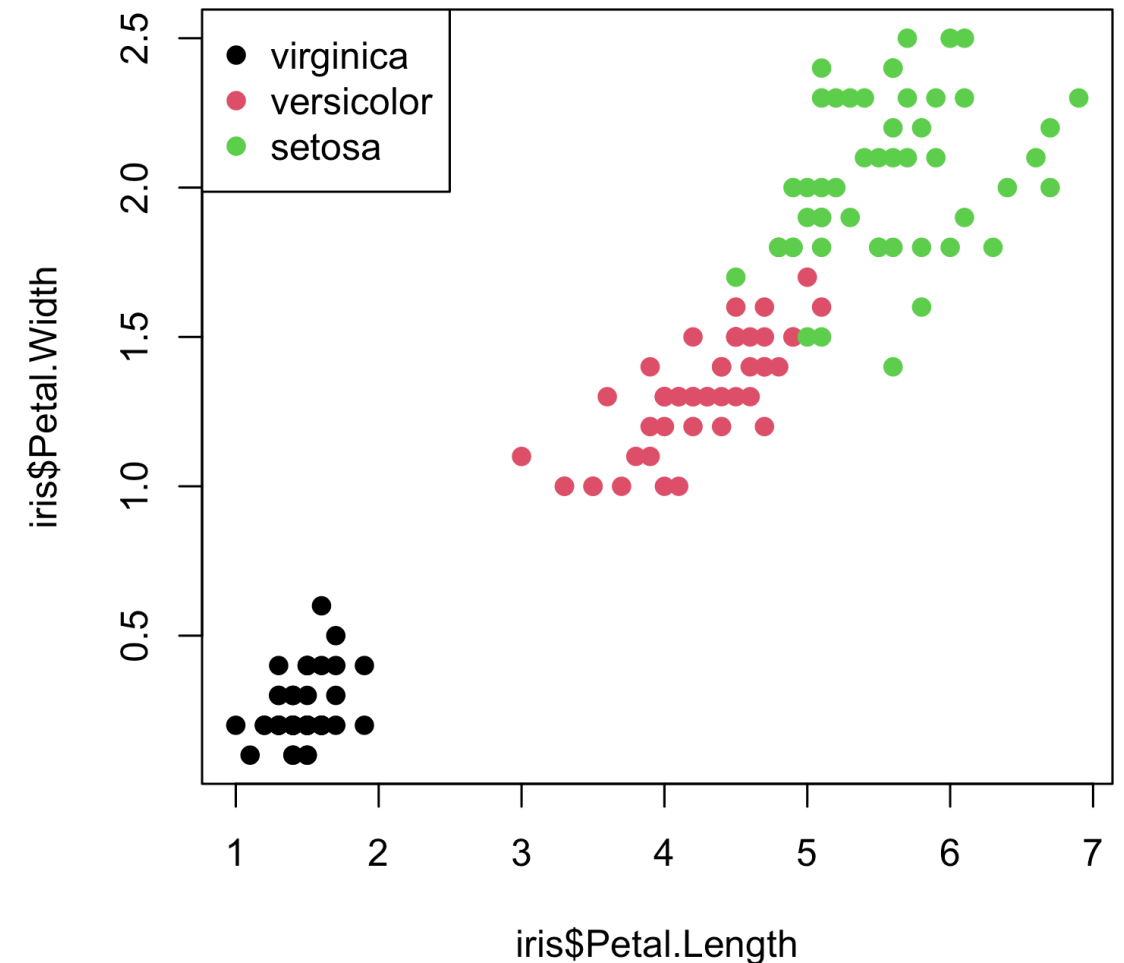
# 3. fermer le fichier
dev.off()
```

- Oublier `dev.off()` : le fichier reste **vide** ou incomplet, et les graphiques suivants s'y écrivent aussi
- `pdf()` peut contenir **plusieurs pages** : un graphique par page jusqu'au `dev.off()`
- Dans RStudio : onglet *Plots* > *Export*, pratique mais **pas reproductible**

Piège : la légende écrite à la main

```
plot(iris$Petal.Length, iris$Petal.Width,  
     col = iris$Species, pch = 19)  
legend("topleft",  
      legend = c("virginica", "versicolor",  
                  "setosa"),  
      col = 1:3, pch = 19)
```

- `col = iris$Species` : R utilise le **numéro** du niveau du facteur : setosa = 1 (noir), versicolor = 2 (rouge), virginica = 3 (vert)
- La légende, elle, est **un texte libre** : R ne vérifie pas qu'elle correspond aux couleurs
- **Aucun message**, et un graphique faux : les petites fleurs noires sont des *setosa*



Correction : `legend = levels(iris$Species)`. Avec `ggplot2`, la légende est construite à partir des données : ce piège disparaît.

👉 À vous : quel graphique ?

Sans exécuter, quel graphique produit chaque appel ?

```
plot(iris$Species)           # 1
plot(iris$Sepal.Length)      # 2
plot(iris$Sepal.Length, iris$Sepal.Width) # 3
plot(iris$Species, iris$Sepal.Length)    # 4
hist(iris$Species)           # 5
```

1. Un **barplot** des effectifs (50 par espèce)
2. Les 150 longueurs **selon leur position** dans le tableau (axe x : de 1 à 150)
3. Un **nuage de points** : largeur selon longueur
4. Trois **boxplots**, un par espèce
5. Une **erreur** : **'x' must be numeric**. Un histogramme découpe un axe **numérique** en intervalles

| 3

LA GRAMMAIRE DE `ggplot2`

Une grammaire des graphiques

« *In brief, the grammar tells us that a graphic maps the **data** to the **aesthetic attributes** (colour, shape, size) of **geometric objects** (points, lines, bars).* »

H. Wickham, *ggplot2: Elegant Graphics for Data Analysis*

Avantages

- Une **grammaire** cohérente (L. Wilkinson, 2005) : les mêmes règles pour tous les graphiques
- Des couches qu'on **ajoute** une à une
- Légendes et axes construits à **partir des données**
- Fait partie du *tidyverse* : attend un tableau rangé

Limites

- Graphiques en 3D
- Réseaux et graphes (voir **igraph**, module réseaux)
- Graphiques interactifs (voir **plotly**)
- Heatmaps annotées : plus simple avec **pheatmap** ou **ComplexHeatmap**

Un graphique en trois ingrédients

```
ggplot(data = fruit, mapping = aes(x = FRUIT, y = QUANTITE)) + geom_col()
```

① les données : un tableau

② le mapping : quelle colonne va sur quelle propriété visuelle

③ le geom : avec quelle forme dessiner

- `ggplot()` crée le graphique **vide** ; chaque **+** ajoute une couche
- Le **+** termine la ligne : une ligne qui commence par **+** est une nouvelle commande, et R ne trace que la première partie
- Les autres couches sont **facultatives** : `stat`, `position`, `scale_*()`, `coord_*()`, `facet_*()`, `theme()`, `labs()`

`ggplot2` est le package ; `ggplot()` est la fonction. Ne pas confondre le **+** de `ggplot2` (ajouter une couche) et le pipe `|>` (passer un résultat à la fonction suivante).

Du tableau au graphique

```
ggplot(fruit, aes(x = FRUIT, y = QUANTITE, fill = COULEUR)) +  
geom_col()
```

fruit.tsv (extrait)

FRUIT	COULEUR	QUANTITE
Fraise	Rouge	10
Banane	Jaune	5
Abricot	Orange	4
Kiwi	Vert	2
Citron	Jaune	9
Citron	Vert	8

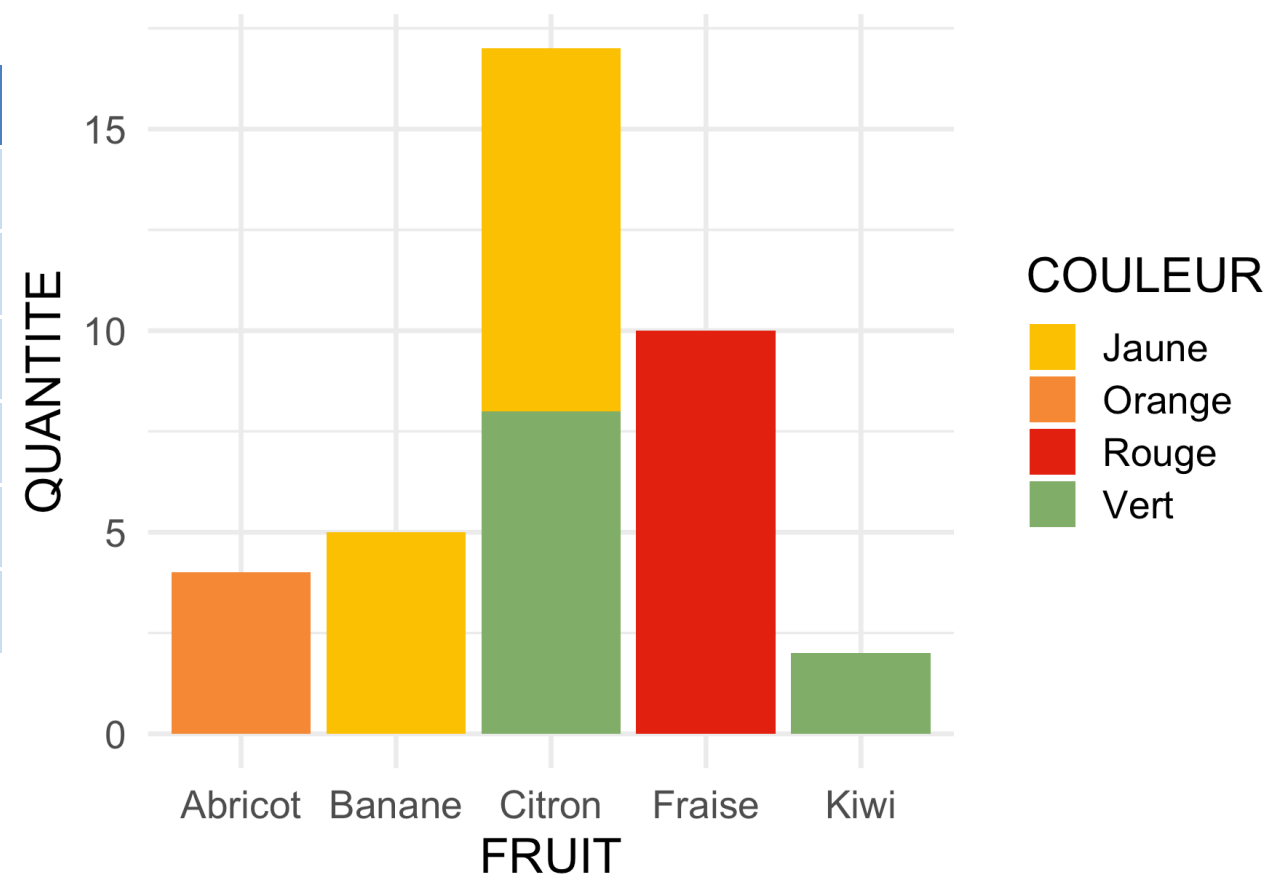
Chaque **ligne** du tableau deviendra **une barre** ; chaque **colonne** peut être reliée à une propriété visuelle.

Du tableau au graphique

```
ggplot(fruit, aes(x = FRUIT, y = QUANTITE, fill = COULEUR)) +  
geom_col()
```

le mapping

x = FRUIT	y = QUANTITE	fill = COULEUR
Fraise	Rouge	10
Banane	Jaune	5
Abricot	Orange	4
Kiwi	Vert	2
Citron	Jaune	9
Citron	Vert	8



aes () répond à **quoi** : quelle colonne sur quel axe, quelle couleur. Le geom répond à **comment** : des barres, des points, des lignes.

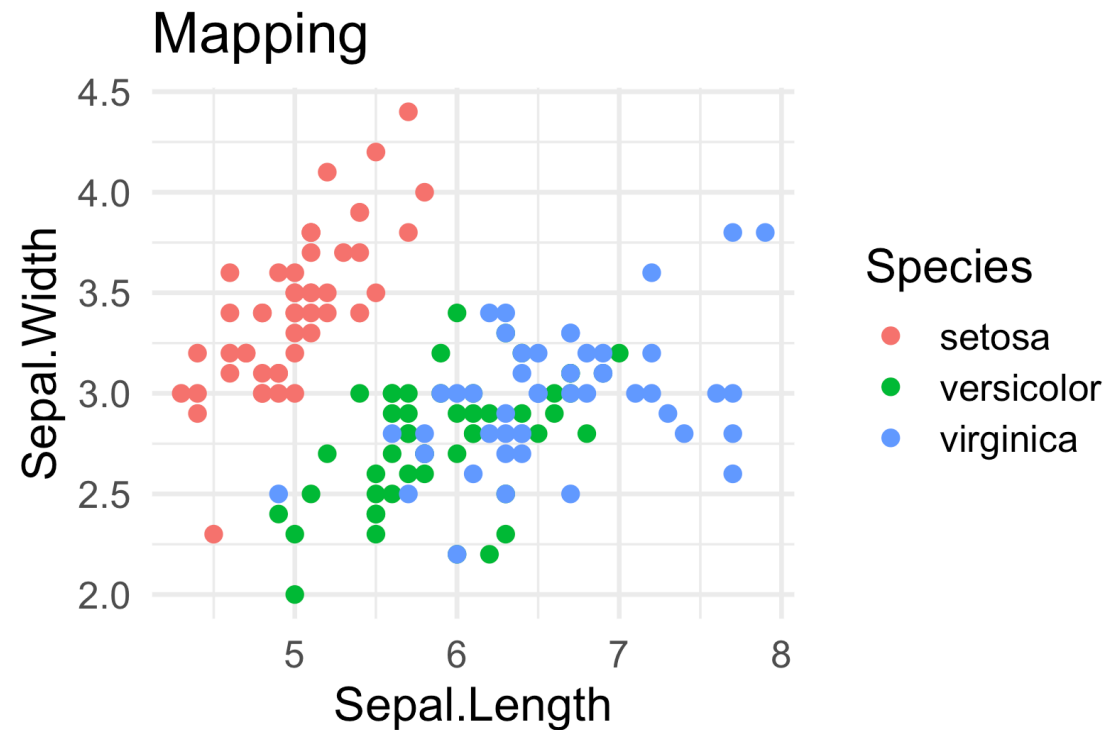
Les *aesthetics*

Les propriétés visuelles qu'on peut relier à une colonne, dans `aes ( )` :

<i>Aesthetic</i>	Propriété	Geoms
<code>x, y</code>	position	tous
<code>color</code> (ou <code>colour</code>)	couleur du contour, des points, des lignes	tous
<code>fill</code>	couleur de remplissage	barres, boxplots, surfaces
<code>size, linewidth</code>	taille des points, épaisseur des lignes	points, lignes
<code>shape</code>	forme des points	points
<code>alpha</code>	transparence (de 0 à 1)	tous
<code>group</code>	quelles lignes forment un même objet (une courbe)	lignes, boxplots
<code>label</code>	le texte affiché	<code>geom_text()</code> , <code>geom_label()</code>

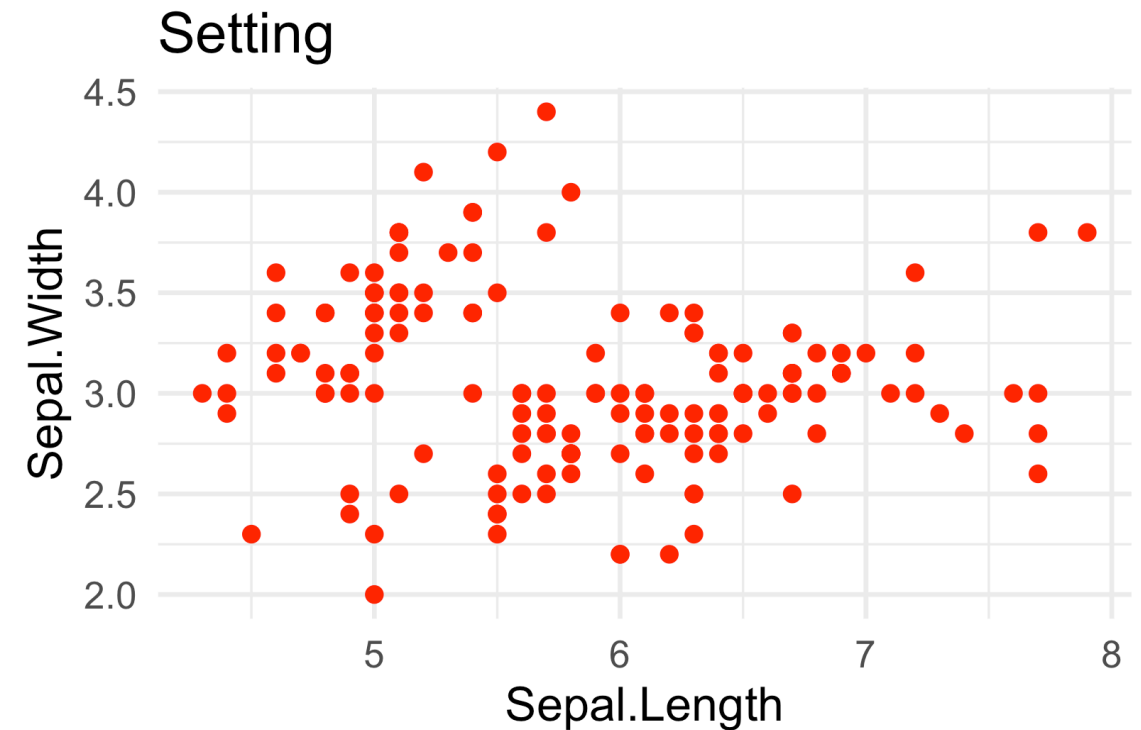
Mapping ou setting ?

```
ggplot(iris, aes(Sepal.Length, Sepal.Width)) +  
  geom_point(aes(color = Species))
```



Mapping, dans `aes()` : la couleur dépend d'une colonne. `ggplot2` choisit les couleurs et crée une légende.

```
ggplot(iris, aes(Sepal.Length, Sepal.Width)) +  
  geom_point(color = "red")
```

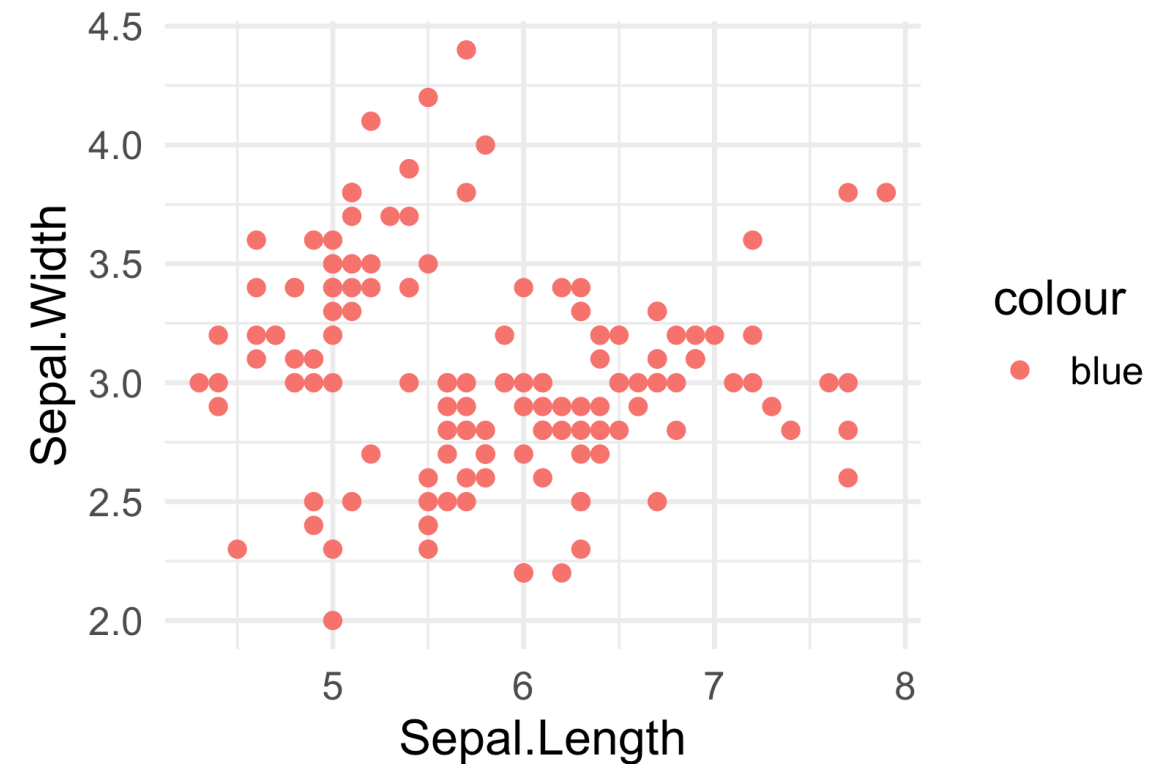


Setting, hors de `aes()` : une valeur fixe pour tous les points. Pas de légende.

Piège : une constante dans `aes()`

```
ggplot(iris, aes(Sepal.Length, Sepal.Width)) +  
  geom_point(aes(color = "blue"))
```

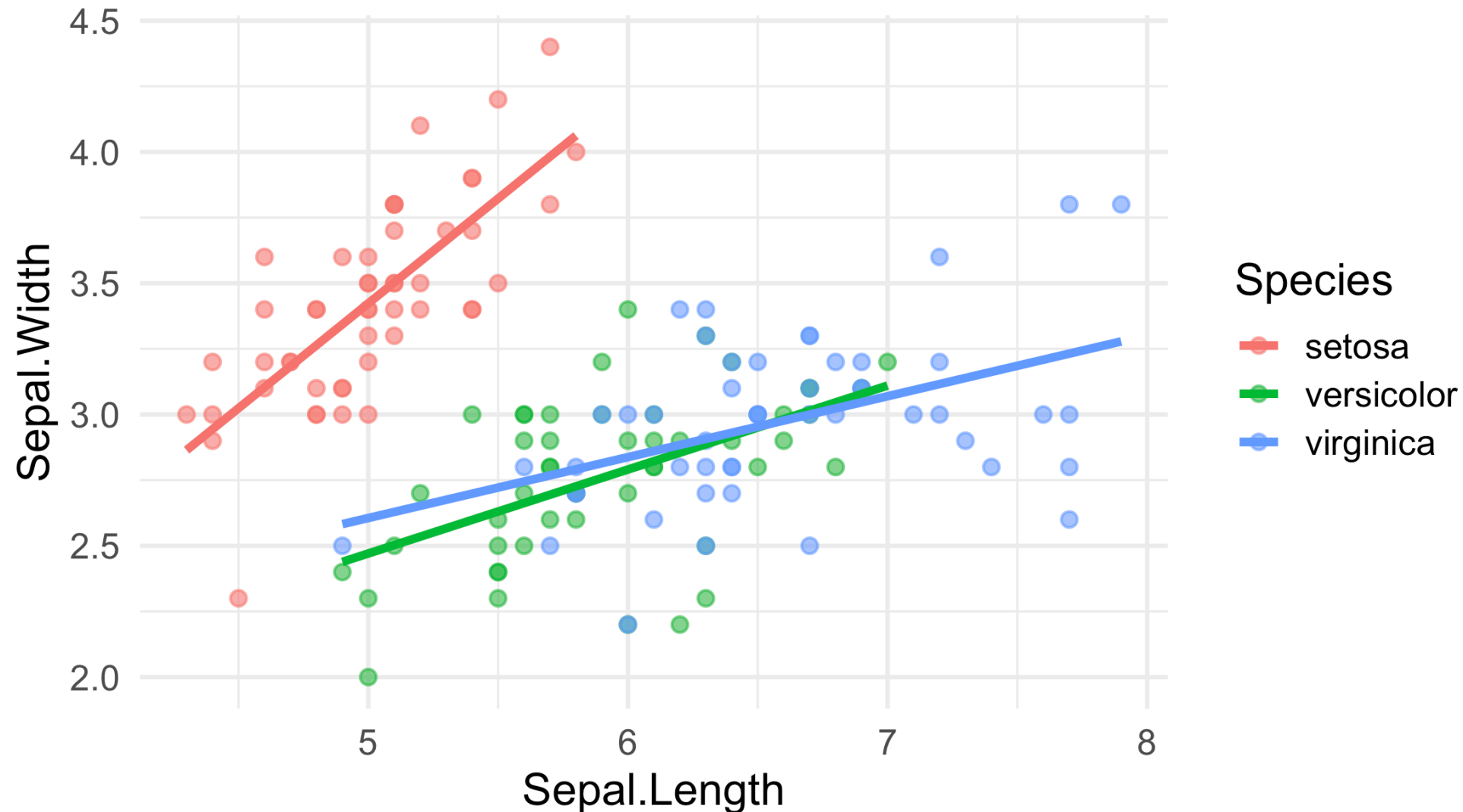
Que va-t-on obtenir ?



- Dans `aes()`, `"blue"` n'est pas une couleur : c'est **une donnée**, une colonne qui vaudrait `"blue"` pour chaque ligne
- `ggplot2` lui attribue la **première couleur de sa palette** (un rouge saumon), et une légende « blue »
- Aucun message. Règle : une **colonne** dans `aes()`, une **valeur fixe** hors de `aes()`

Superposer des couches

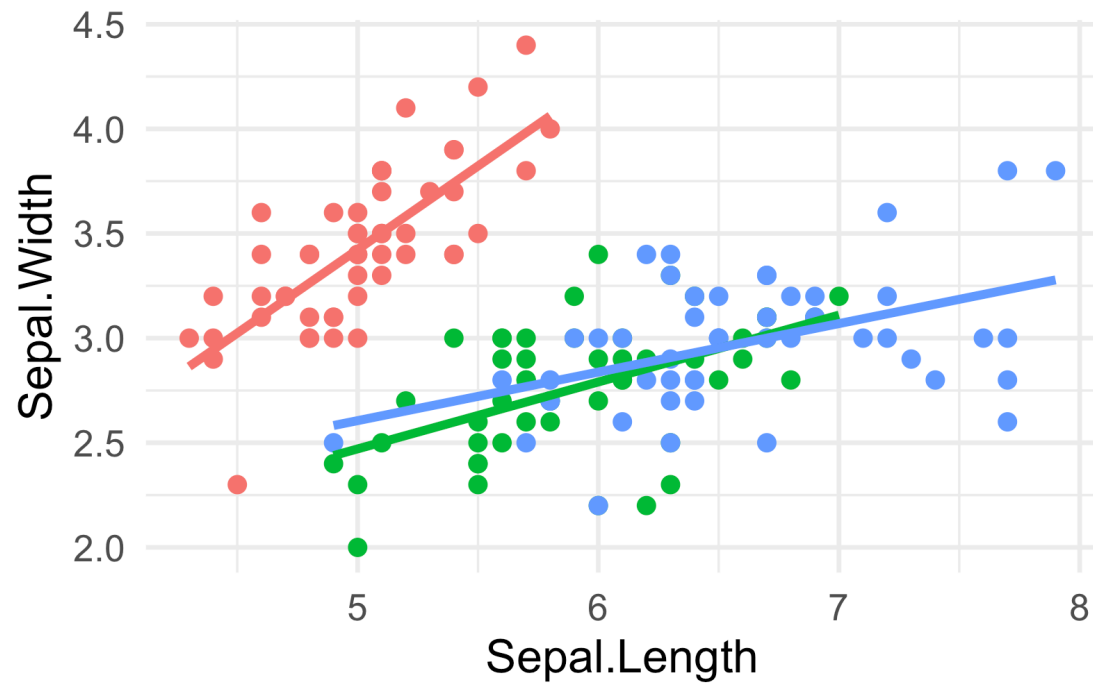
```
ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, color = Species)) +  
  geom_point(alpha = 0.6) + # couche 1 : les points  
  geom_smooth(method = "lm", se = FALSE) # couche 2 : une droite de régression par espèce
```



Les couches se dessinent **dans l'ordre** : la dernière passe par-dessus les autres.

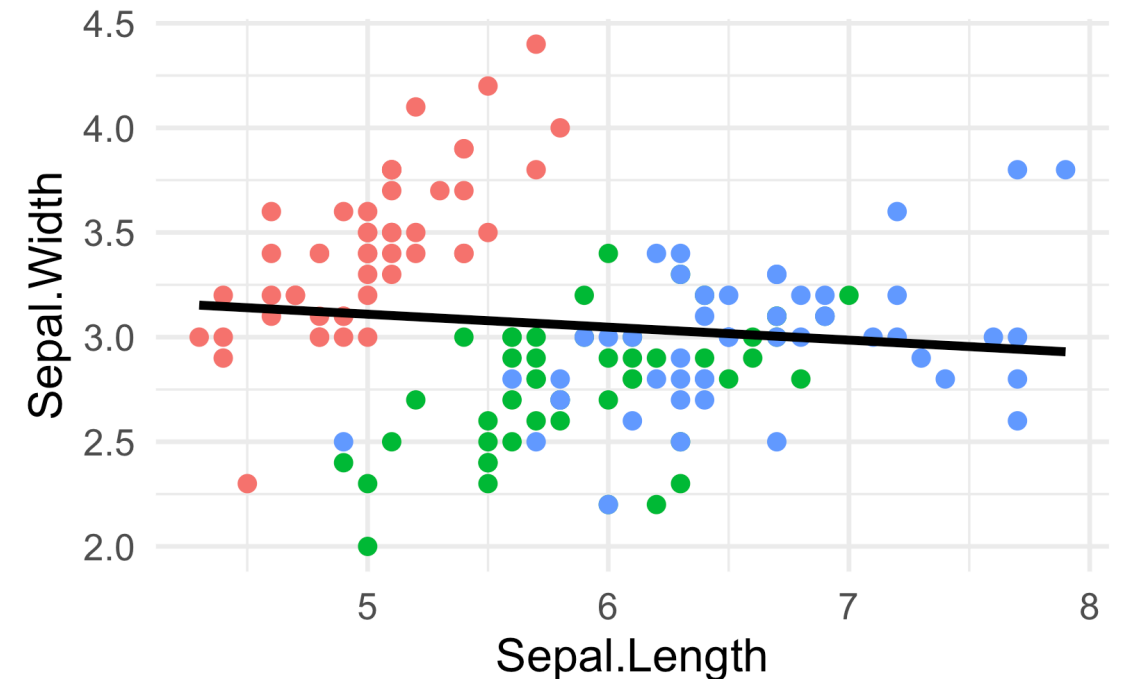
Où placer **aes()** ?

```
ggplot(iris, aes(Sepal.Length, Sepal.Width,  
                 color = Species)) +  
  geom_point() +  
  geom_smooth(method = "lm", se = FALSE)
```



Dans **ggplot()** : l'*aesthetic* vaut pour **toutes les couches** : trois droites.

```
ggplot(iris, aes(Sepal.Length, Sepal.Width)) +  
  geom_point(aes(color = Species)) +  
  geom_smooth(method = "lm", se = FALSE)
```



Dans un geom : seulement **cette couche** : une seule droite, pour toutes les fleurs.

Les deux graphiques sont justes ; ils ne répondent pas à la même question. La pente globale est **négative**, les pentes par espèce **positives** (un paradoxe de Simpson, module 9).

Les *geoms*

Le choix du geom dépend du **nombre** et du **type** des variables à représenter :

Variables	Geoms
1 numérique	<code>geom_histogram()</code> , <code>geom_density()</code> , <code>geom_boxplot()</code>
1 catégorielle	<code>geom_bar()</code> (compte les lignes)
1 catégorielle + 1 numérique	<code>geom_boxplot()</code> , <code>geom_violin()</code> , <code>geom_jitter()</code> , <code>geom_col()</code>
2 numériques	<code>geom_point()</code> , <code>geom_smooth()</code> , <code>geom_line()</code> (si x est ordonné : temps)
2 catégorielles	<code>geom_count()</code> , <code>geom_tile()</code>
Repères	<code>geom_hline()</code> , <code>geom_vline()</code> , <code>geom_abline()</code> , <code>geom_text()</code>

Tous les geoms et leurs *aesthetics* : l'aide-mémoire *Data visualization with ggplot2* (Posit).

L'aide-mémoire

GRAPHICAL PRIMITIVES

```
a <- ggplot(economics, aes(date, unemploy))  
b <- ggplot(seals, aes(x = long, y = lat))
```

 **a + geom_blank()** and **a + expand_limits()**
Ensure limits include values across all plots.

 **b + geom_curve()**(aes(yend = lat + 1, xend = long + 1, curvature = 1) - x, yend, y, yend, alpha, angle, color, curvature, linetype, size)

 **a + geom_path()**(lineend = "butt", linejoin = "round", linemitre = 1) - x, y, alpha, color, group, linetype, size

 **a + geom_polygon()**(aes(alpha = 50)) - x, y, alpha, color, fill, group, subgroup, linetype, size

 **b + geom_rect()**(aes(xmin = long, ymin = lat, xmax = long + 1, ymax = lat + 1)) - xmax, xmin, ymax, ymin, alpha, color, fill, linetype, size

 **a + geom_ribbon()**(aes(ymin = unemploy - 900, ymax = unemploy + 900)) - x, ymax, ymin, alpha, color, fill, group, linetype, size

LINE SEGMENTS

common aesthetics: x, y, alpha, color, linetype, size

 **b + geom_abline()**(aes(intercept = 0, slope = 1))
 **b + geom_hline()**(aes(yintercept = lat))
 **b + geom_vline()**(aes(xintercept = long))

b + geom_segment()(aes(yend = lat + 1, xend = long + 1))
b + geom_spoke()(aes(angle = 1:1155, radius = 1))

ONE VARIABLE continuous

```
c <- ggplot(mpg, aes(hwy)); c2 <- ggplot(mpg)
```

 **c + geom_area()**(stat = "bin")
x, y, alpha, color, fill, linetype, size

 **c + geom_density()**(kernel = "gaussian")
x, y, alpha, color, fill, group, linetype, size, weight

 **c + geom_dotplot()**
x, y, alpha, color, fill

 **c + geom_freqpoly()**
x, y, alpha, color, group, linetype, size

 **c + geom_histogram()**(binwidth = 5)
x, y, alpha, color, fill, linetype, size, weight

 **c2 + geom_qq()**(aes(sample = hwy))
x, y, alpha, color, fill, linetype, size, weight

discrete

```
d <- ggplot(mpg, aes(fl))
```

 **d + geom_bar()**
x, alpha, color, fill, linetype, size, weight

TWO VARIABLES

both continuous

```
e <- ggplot(mpg, aes(cty, hwy))
```

 **e + geom_label()**(aes(label = cty), nudge_x = 1, nudge_y = 1) - x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

 **e + geom_point()**
x, y, alpha, color, fill, shape, size, stroke

 **e + geom_quantile()**
x, y, alpha, color, group, linetype, size, weight

 **e + geom_rug()**(sides = "bl")
x, y, alpha, color, linetype, size

 **e + geom_smooth()**(method = lm)
x, y, alpha, color, fill, group, linetype, size, weight

 **e + geom_text()**(aes(label = cty), nudge_x = 1, nudge_y = 1) - x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

one discrete, one continuous

```
f <- ggplot(mpg, aes(class, hwy))
```

 **f + geom_col()**
x, y, alpha, color, fill, group, linetype, size

 **f + geom_boxplot()**
x, y, lower, middle, upper, ymax, ymin, alpha, color, fill, group, linetype, shape, size, weight

 **f + geom_dotplot()**(binaxis = "y", stackdir = "center")
x, y, alpha, color, fill, group

 **f + geom_violin()**(scale = "area")
x, y, alpha, color, fill, group, linetype, size, weight

both discrete

```
g <- ggplot(diamonds, aes(cut, color))
```

 **g + geom_count()**
x, y, alpha, color, fill, shape, size, stroke

 **e + geom_jitter()**(height = 2, width = 2)
x, y, alpha, color, fill, shape, size

THREE VARIABLES

```
sealsSz <- with(seals, sqrt(delta_long^2 + delta_lat^2)); l <- ggplot(seals, aes(long, lat))
```

 **l + geom_contour()**(aes(z = z))
x, y, z, alpha, color, group, linetype, size, weight

 **l + geom_contour_filled()**(aes(fill = z))
x, y, alpha, color, fill, group, linetype, size, subgroup

continuous bivariate distribution

```
h <- ggplot(diamonds, aes(carat, price))
```

 **h + geom_bin2d()**(binwidth = c(0.25, 500))
x, y, alpha, color, fill, linetype, size, weight

 **h + geom_density_2d()**
x, y, alpha, color, group, linetype, size

 **h + geom_hex()**
x, y, alpha, color, fill, size

continuous function

```
i <- ggplot(economics, aes(date, unemploy))
```

 **i + geom_area()**
x, y, alpha, color, fill, linetype, size

 **i + geom_line()**
x, y, alpha, color, group, linetype, size

 **i + geom_step()**(direction = "hv")
x, y, alpha, color, group, linetype, size

visualizing error

```
df <- data.frame(grp = c("A", "B"), fit = 4:5, se = 1:2)  
j <- ggplot(df, aes(grp, fit, ymin = fit - se, ymax = fit + se))
```

 **j + geom_crossbar()**(fatten = 2) - x, y, ymax, ymin, alpha, color, fill, group, linetype, size

 **j + geom_errorbar()** - x, ymax, ymin, alpha, color, group, linetype, size, width
Also **geom_errorbarh()**.

 **j + geom_linerange()**
x, ymin, ymax, alpha, color, group, linetype, size

 **j + geom_pointrange()** - x, y, ymin, ymax, alpha, color, fill, group, linetype, shape, size

maps

```
data <- data.frame(murder = USArrests$Murder,  
state = tolower(rownames(USArrests)))
```

```
map <- map_data("state")
```

```
k <- ggplot(data, aes(fill = murder))
```

 **k + geom_map()**(aes(map_id = state), map = map) + **expand_limits**(x = map\$long, y = map\$lat)
map_id, alpha, color, fill, linetype, size

👉 À vous : prédire

Sans exécuter : un graphique (lequel ?) ou une erreur ?

```
ggplot(iris, aes(Species, Sepal.Length)) + geom_point()           # 1
ggplot(iris, aes(Sepal.Length)) + geom_point()                   # 2
ggplot(iris, aes(Sepal.Length, Sepal.Width)) + geom_point(color = Species) # 3
ggplot(iris, aes(Sepal.Length, Sepal.Width))                     # 4
ggplot(iris, aes(Sepal.Length, Sepal.Width))                     # 5
  + geom_point()
```

1. Trois **colonnes de points** superposés, une par espèce (d'où **geom_jitter()**)
2. Erreur : **geom_point() requires the following missing aesthetics: y**
3. Erreur : **object 'Species' not found**. Hors de **aes()**, R cherche une **variable Species**, pas une colonne
4. Un graphique **vide**, avec les axes : il n'y a aucune couche de geom
5. Un graphique vide, **puis** une erreur sur la ligne **+ geom_point()** : le **+** doit **terminer** la ligne

TP1

| 4

AJUSTER UN GRAPHIQUE

geom_bar() compte les lignes

```
ggplot(fruit, aes(x = COULEUR)) + geom_bar()
```

fruit (extrait), trié par couleur

FRUIT	COULEUR	QUANTITE
Banane	Jaune	5
Citron	Jaune	9
Abricot	Orange	4
Fraise	Rouge	10
Kiwi	Vert	2
Citron	Vert	8

On ne donne que **x** : la hauteur de chaque barre est **calculée** par `geom_bar()`.

geom_bar() compte les lignes

```
ggplot(fruit, aes(x = COULEUR)) + geom_bar()
```

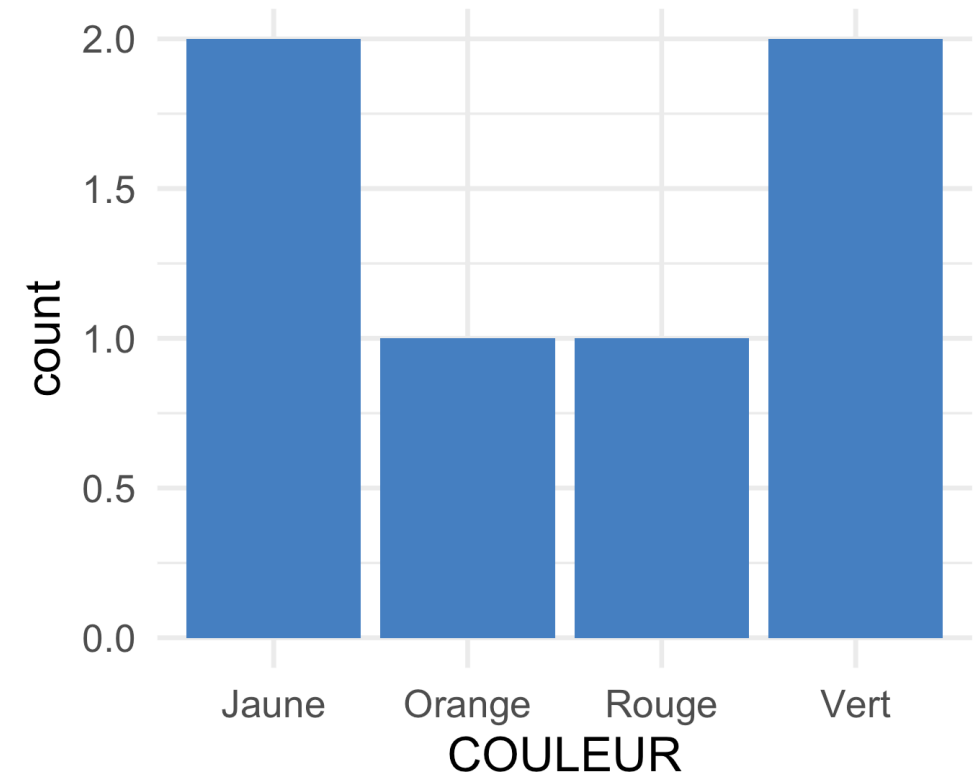
6 lignes

FRUIT	COULEUR	QUANTITE
Banane	Jaune	5
Citron	Jaune	9
Abricot	Orange	4
Fraise	Rouge	10
Kiwi	Vert	2
Citron	Vert	8



stat = "count"

COULEUR	n
Jaune	2
Orange	1
Rouge	1
Vert	2



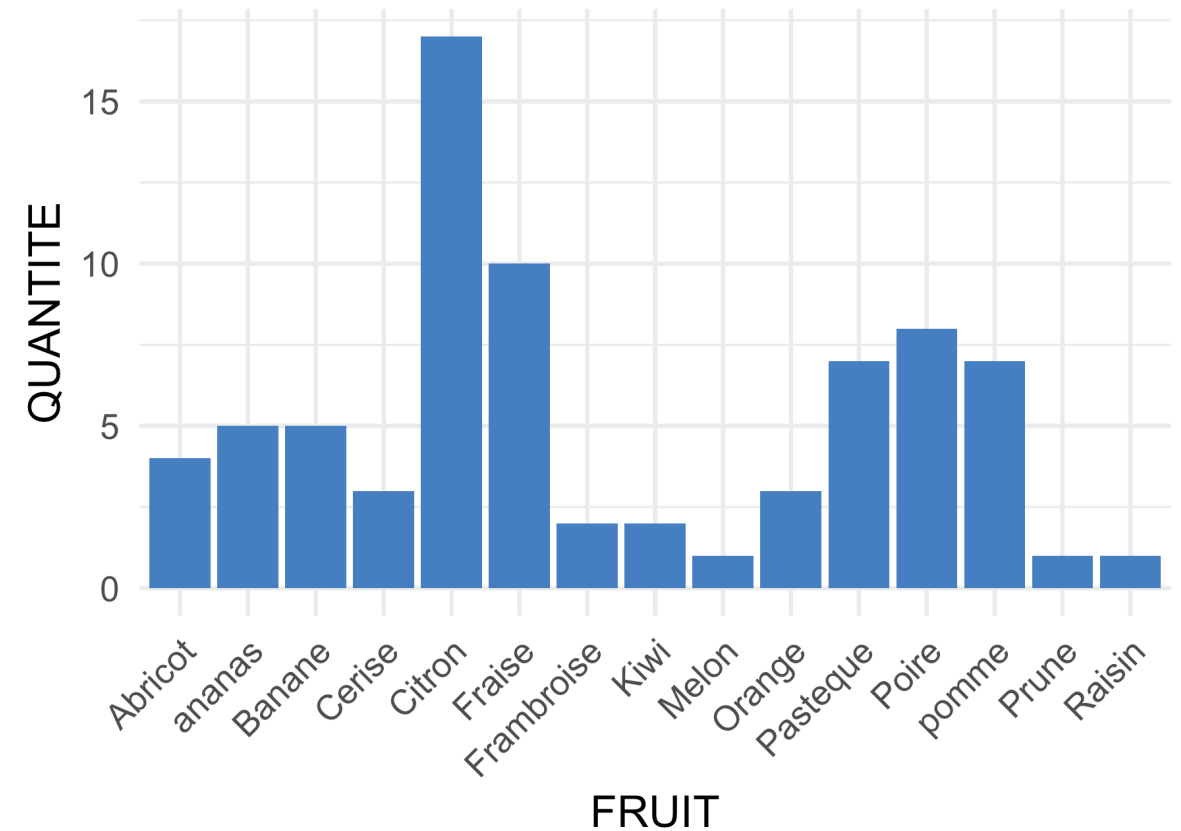
Chaque geom a une **statistique** par défaut : `geom_bar()` utilise `stat = "count"`, `geom_histogram()` `stat = "bin"`, `geom_boxplot()` `stat = "boxplot"`, `geom_point()` `stat = "identity"` (les valeurs telles quelles).

geom_col() : la hauteur est dans les données

```
ggplot(fruit, aes(x = FRUIT, y = QUANTITE)) +  
  geom_col()
```

```
# équivalent (forme des anciens scripts) :  
ggplot(fruit, aes(x = FRUIT, y = QUANTITE)) +  
  geom_bar(stat = "identity")
```

- **geom_bar()** : une variable, il compte
- **geom_col()** : **x** et **y**, il trace **y** tel quel
- **geom_bar(aes(x, y))** sans **stat = "identity"** : erreur **must only have an x or y aesthetic**



Piège : plusieurs lignes pour une même barre

```
ggplot(fruit, aes(x = FRUIT, y = QUANTITE)) + geom_col()
```

fruit (extrait)

FRUIT	COULEUR	QUANTITE
Fraise	Rouge	10
Banane	Jaune	5
Abricot	Orange	4
Kiwi	Vert	2
Citron	Jaune	9
Citron	Vert	8

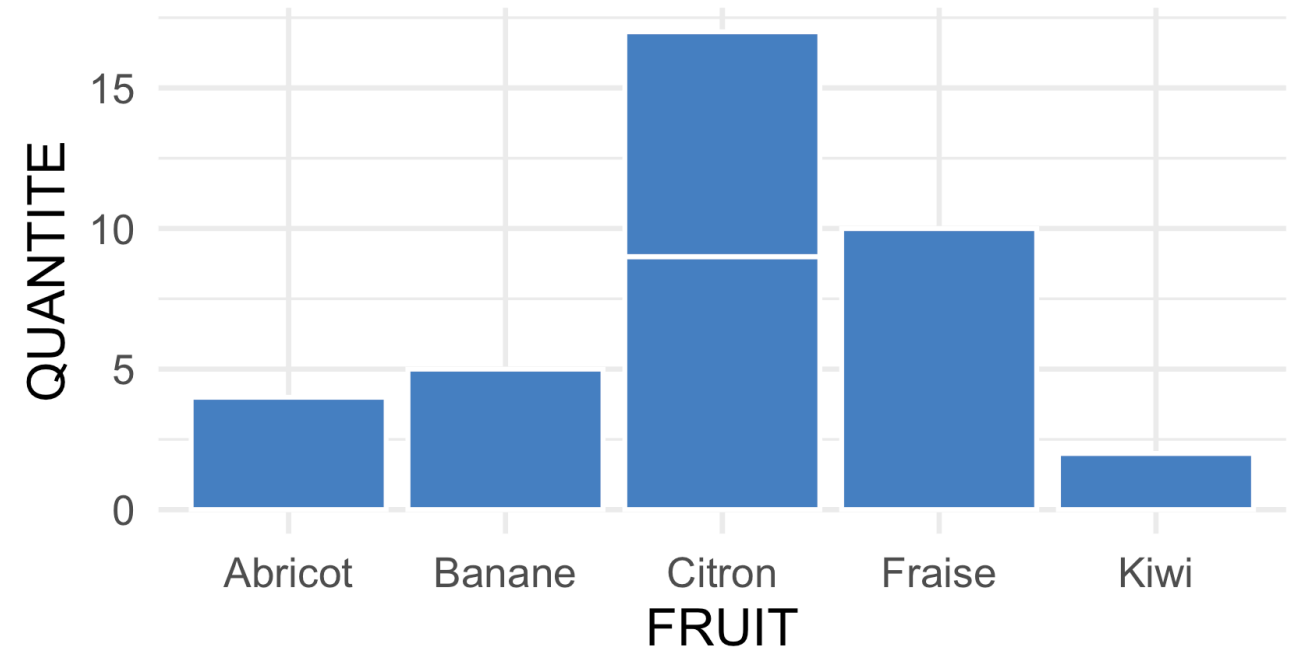
Quelle sera la hauteur de la barre « Citron » ?

Piège : plusieurs lignes pour une même barre

```
ggplot(fruit, aes(x = FRUIT, y = QUANTITE)) + geom_col()
```

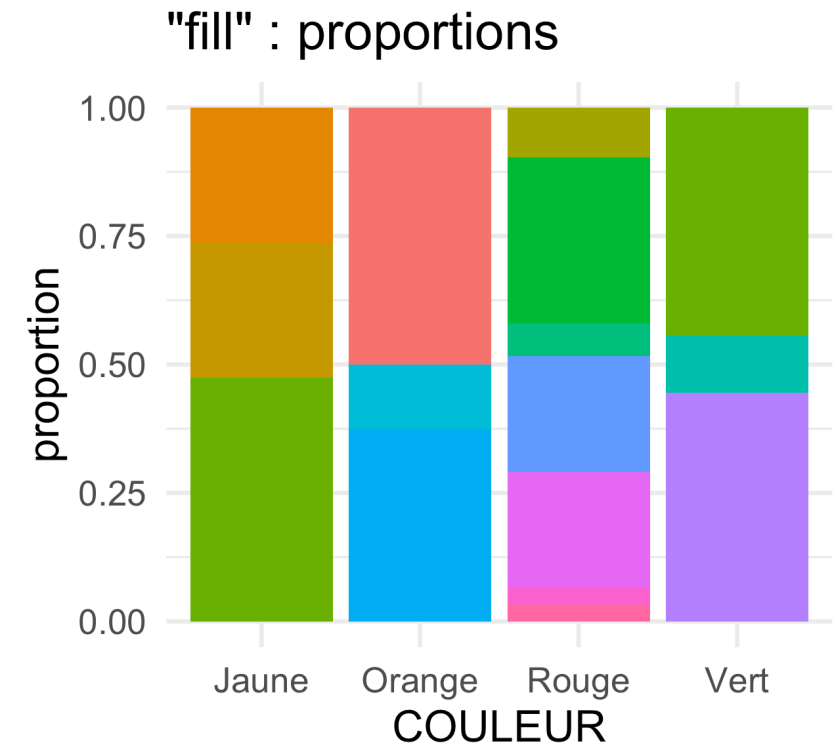
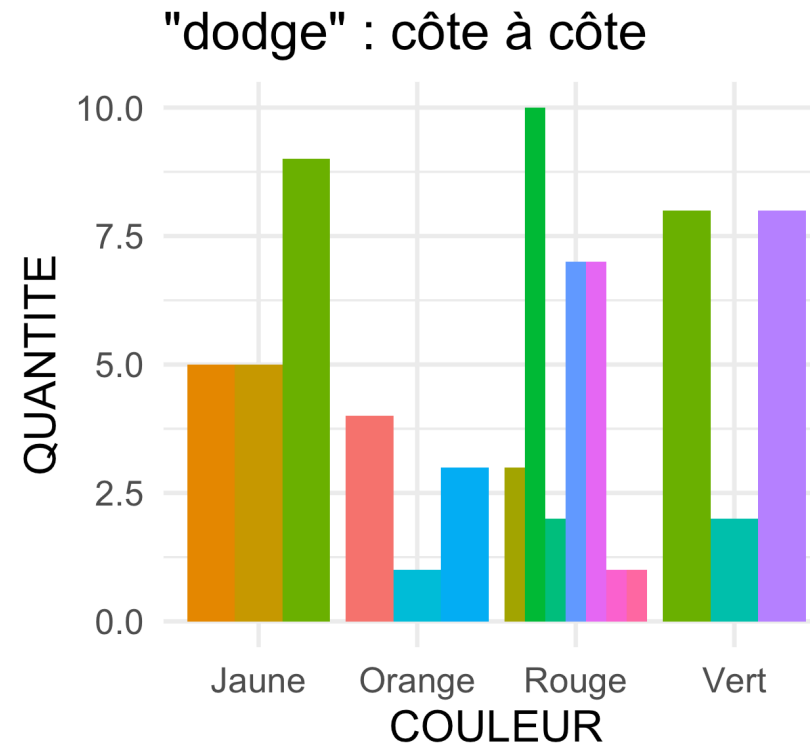
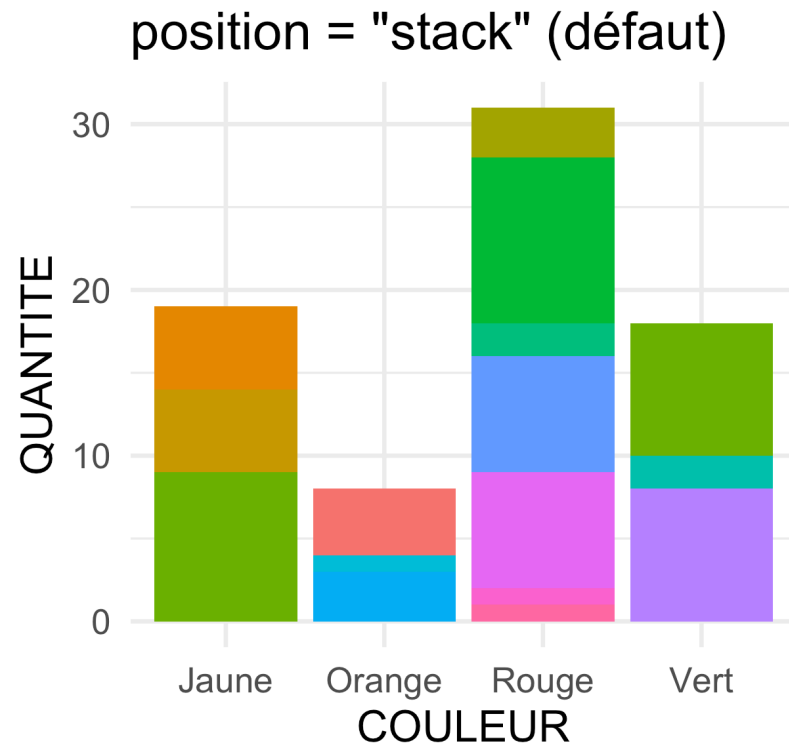
2 lignes Citron

FRUIT	COULEUR	QUANTITE
Fraise	Rouge	10
Banane	Jaune	5
Abricot	Orange	4
Kiwi	Vert	2
Citron	Jaune	9
Citron	Vert	8



- `geom_col()` empile les lignes de même `x` : la barre vaut $9 + 8 = 17$, la somme. Aucun message
- Pour une moyenne, la calculer d'abord : `summarise(moy = mean(QUANTITE), .by = FRUIT)`
- Ici, le trait blanc trahit l'empilement ; sans contour, rien ne se voit

Les positions : **stack**, **dodge**, **fill**

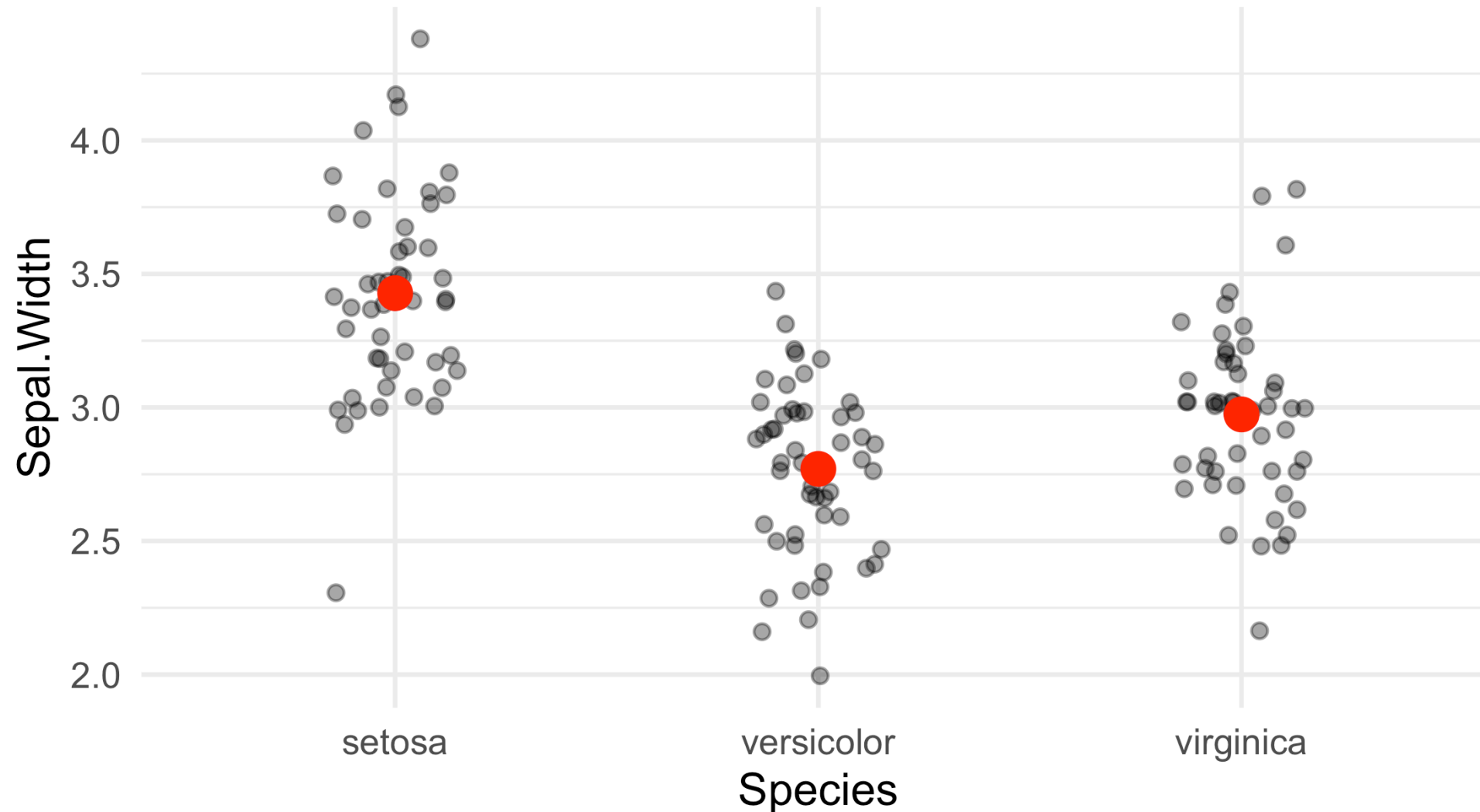


```
ggplot(fruit, aes(COULEUR, QUANTITE, fill = FRUIT)) + geom_col(position = "dodge")
```

Avec **"fill"**, l'axe y ne montre plus des quantités mais des **proportions** (chaque barre vaut 1) : renommer l'axe.

Résumer dans le graphique : `stat_summary()`

```
ggplot(iris, aes(x = Species, y = Sepal.Width)) +  
  geom_jitter(width = 0.15, alpha = 0.4) +  
  stat_summary(fun = mean, geom = "point", color = "red", size = 5) # toutes les fleurs  
                                                                    # moyenne par espèce
```

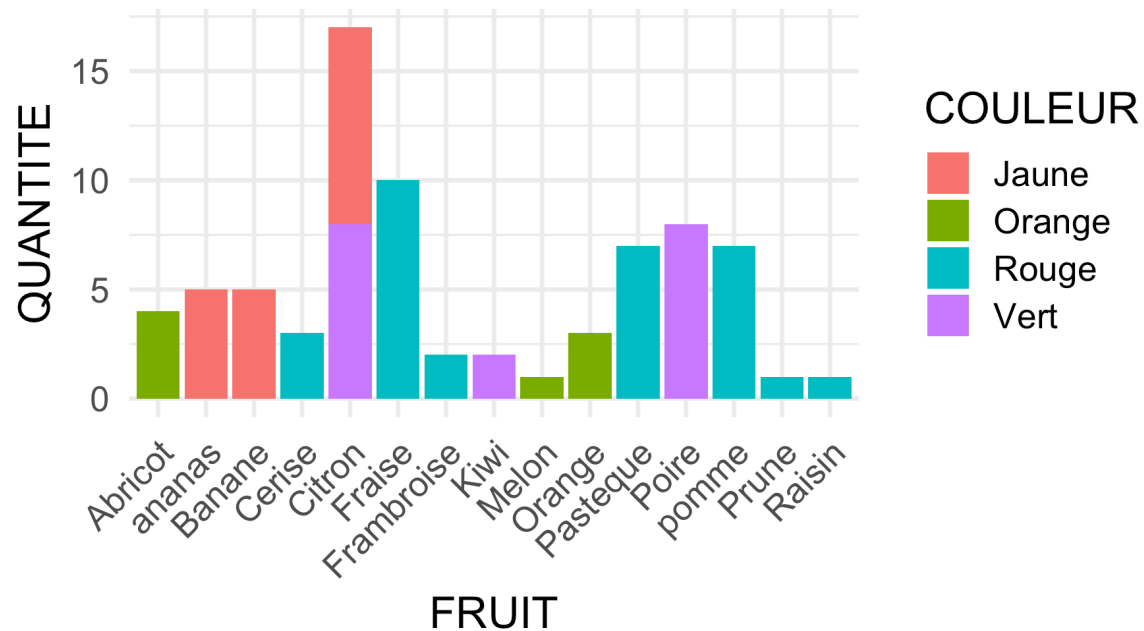


Montrer les données et leur résumé : c'est souvent le graphique le plus honnête.

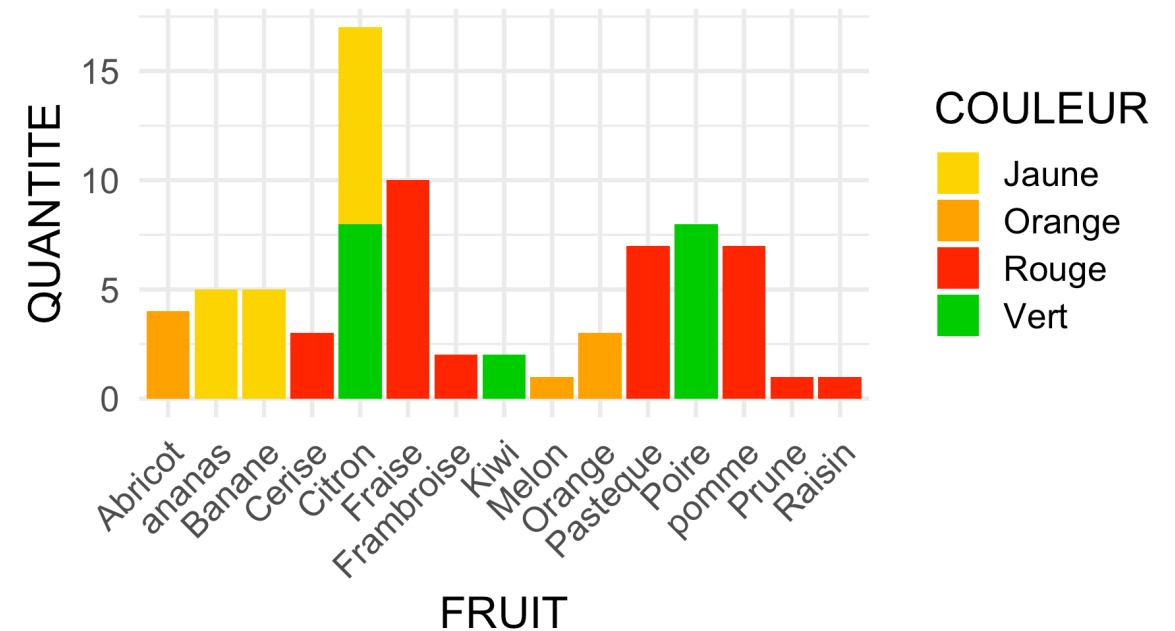
Les échelles : `scale_*()`

Une échelle contrôle **comment** une valeur devient une **propriété visuelle**, et l'axe ou la légende qui va avec.

```
ggplot(fruit, aes(FRUIT, QUANTITE,  
                  fill = COULEUR)) +  
  geom_col()
```



```
... + scale_fill_manual(values = c(  
  Jaune = "gold", Orange = "orange",  
  Rouge = "red", Vert = "green3"))
```



`ggplot2` ne lit pas le **contenu** : « Jaune » reçoit la première couleur de la palette (rouge saumon), « Rouge » un bleu.

Un vecteur **nommé** : chaque catégorie reçoit sa couleur, quel que soit l'ordre.

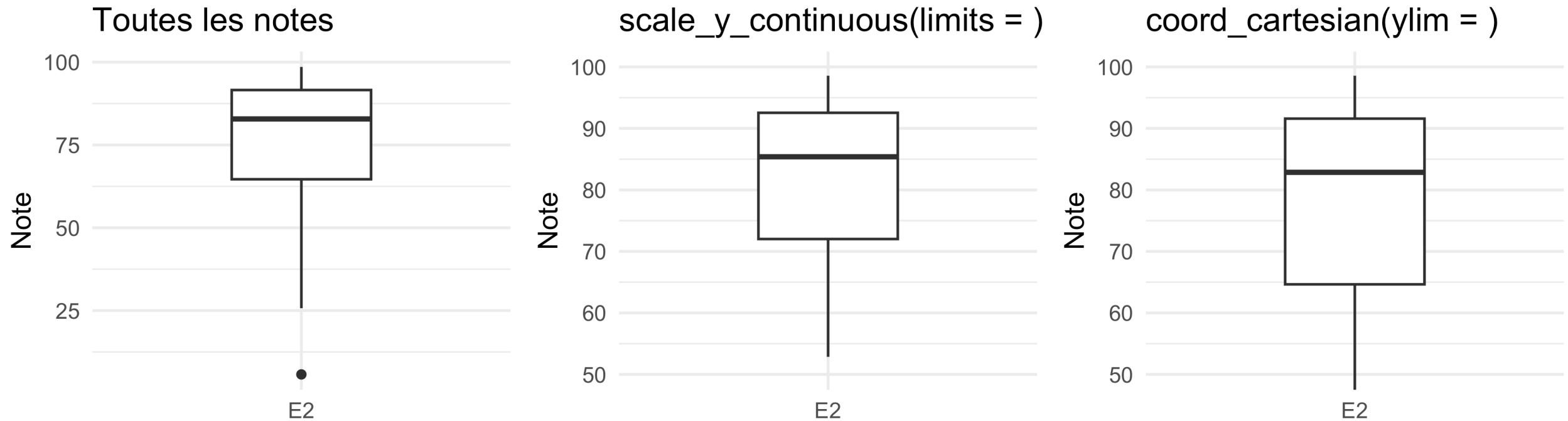
Les familles d'échelles

Échelle	Usage
<code>scale_color_manual()</code> , <code>scale_fill_manual()</code>	couleurs choisies à la main (vecteur nommé)
<code>scale_color_viridis_d()</code> , <code>scale_fill_brewer()</code>	palettes prêtes à l'emploi, lisibles par les daltoniens
<code>scale_color_gradient(low, high)</code>	une variable continue : un dégradé
<code>scale_color_gradient2(low, mid, high, midpoint)</code>	dégradé divergent (<i>log2 fold change</i> , corrélations)
<code>scale_x_continuous(breaks =)</code> , <code>scale_y_continuous(labels =)</code>	graduations, étiquettes des axes
<code>scale_x_log10()</code> , <code>scale_y_log10()</code>	axe logarithmique
<code>scale_x_discrete(limits =)</code>	ordre et sélection des catégories

Le nom se lit : `scale_` + *aesthetic* + `_` + type d'échelle.

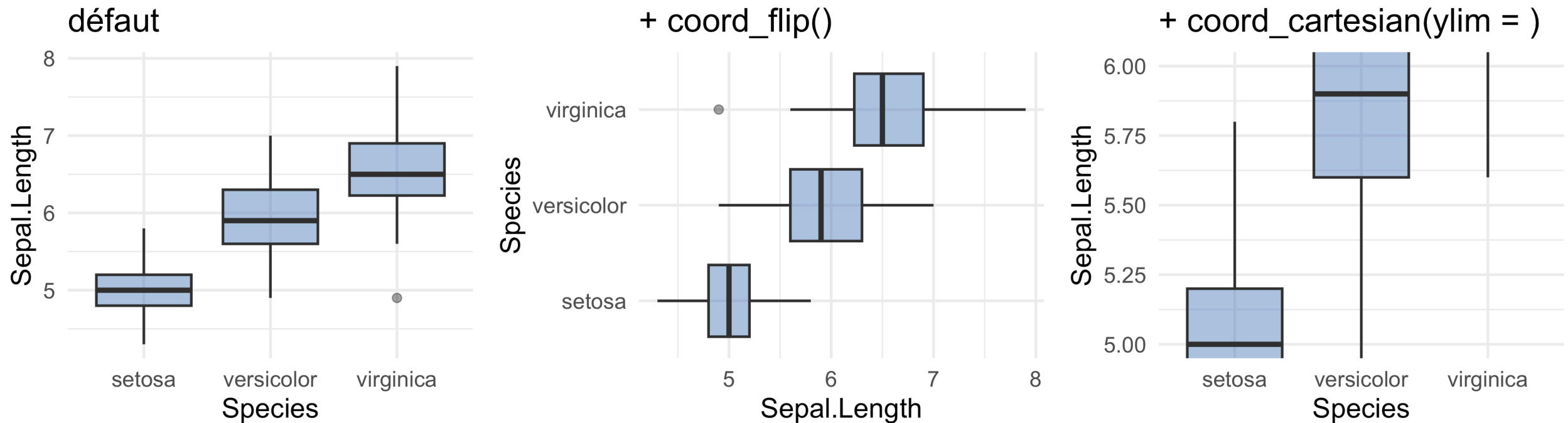
Piège : **limits** retire des données

Les notes de l'examen E2, sur 100. On veut « zoomer » sur la partie haute :



- **limits** dans une échelle **supprime** les 11 notes sous 50 **avant** de calculer le boxplot : la médiane passe de 82.86 à 85.39. Seul un *warning* le signale : « Removed 11 rows containing non-finite outside the scale range »
- **coord_cartesian()** **zoome** : toutes les données sont gardées, le boxplot est juste
- Même piège avec **xlim()**, **ylim()** : ce sont des raccourcis pour **limits**

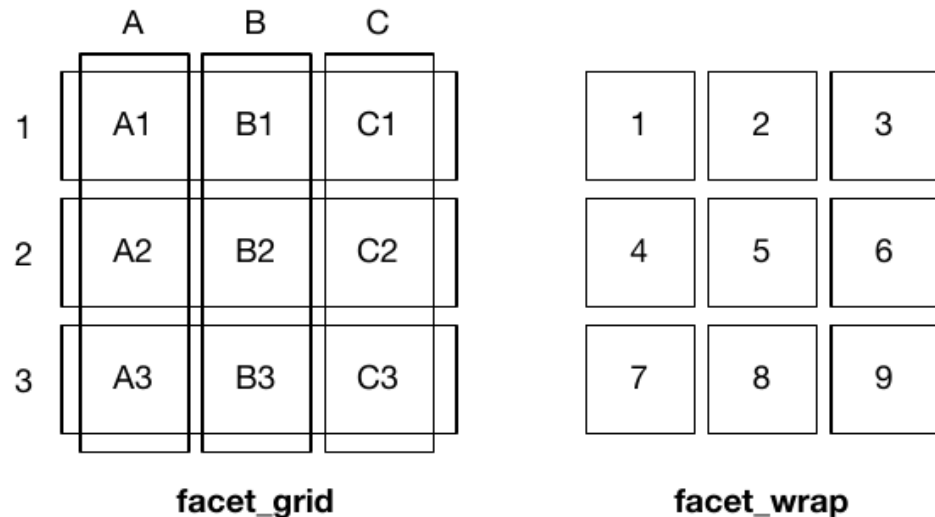
Les systèmes de coordonnées



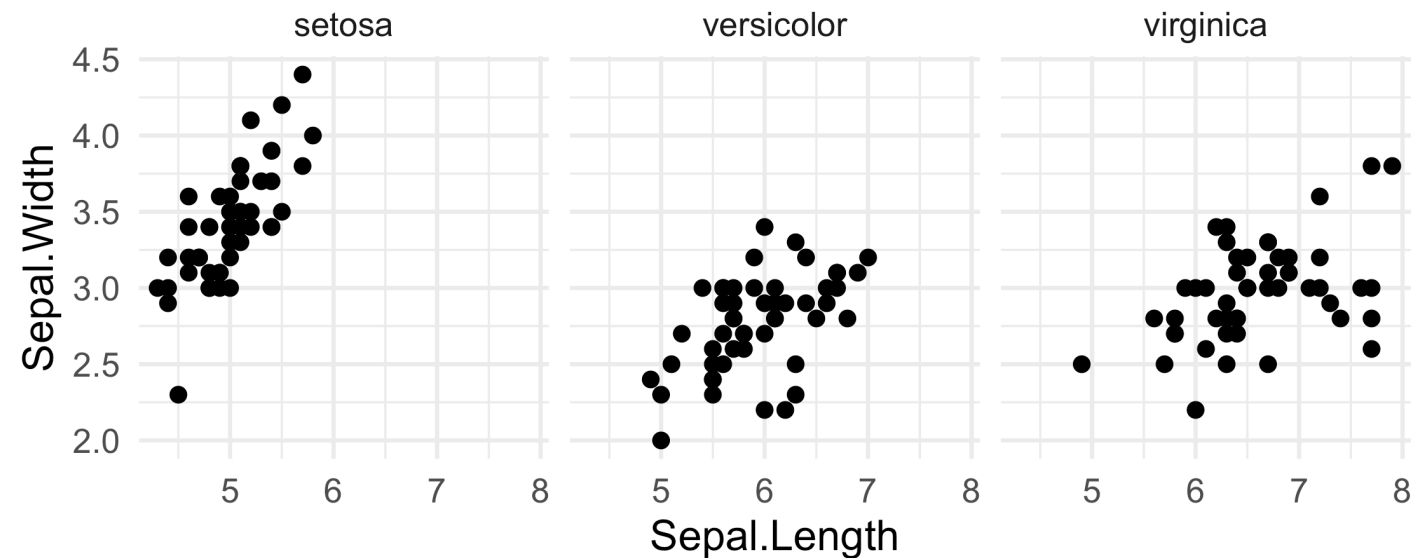
- `coord_cartesian(xlim, ylim)` : **zoomer** sans retirer de données
- `coord_flip()` : échanger x et y (étiquettes longues : on peut aussi écrire `aes(y = categorie)`)
- `coord_fixed()` : même échelle sur les deux axes (un nuage de points d'une mesure contre elle-même)

Les facettes

Un graphique par **sous-ensemble** des données :



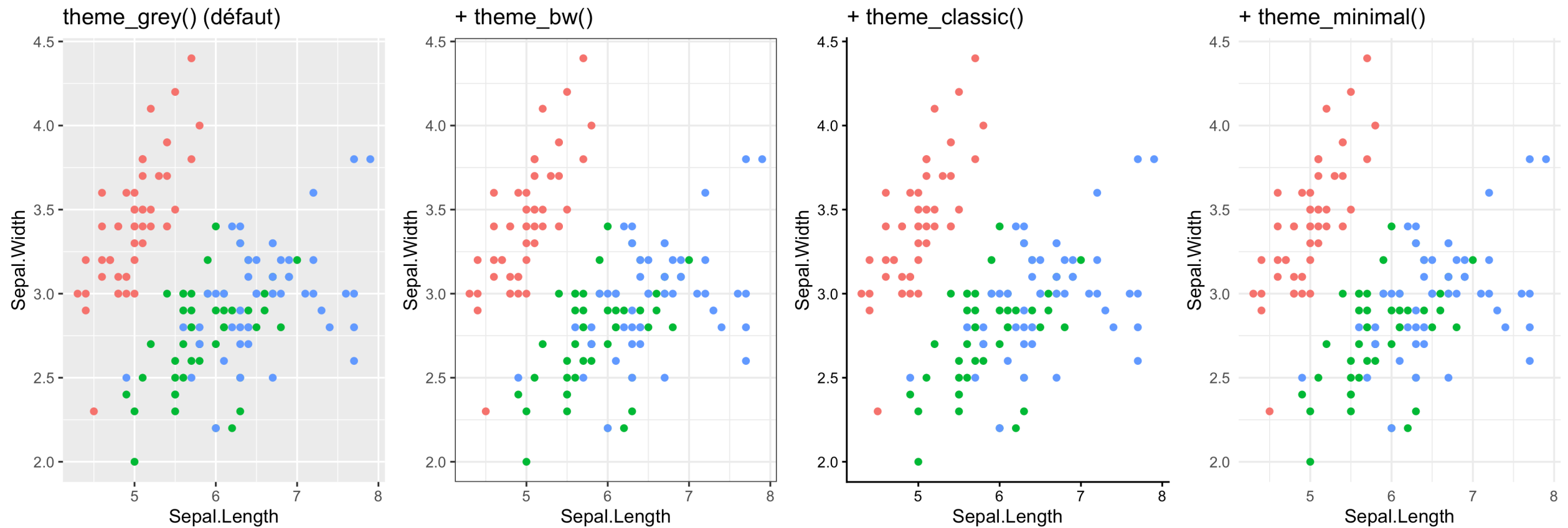
```
ggplot(iris, aes(Sepal.Length, Sepal.Width)) +  
  geom_point() +  
  facet_wrap(~ Species)
```



```
ggplot(diamonds, aes(carat, price)) +  
  geom_point() + facet_grid(cut ~ color)
```

- **facet_wrap(~ var)** : un panneau par valeur, qui passe à la ligne
- **facet_grid(lignes ~ colonnes)** : une grille selon deux variables
- **scales = "free_y"** : un axe y propre à chaque panneau ; **attention** aux comparaisons entre panneaux

Les thèmes



Le thème change l'apparence (fond, grille, polices), jamais les données.

`theme_set(theme_bw())` en début de script : pour tous les graphiques.

Titres, légendes, sauvegarde

```
ggplot(iris, aes(Sepal.Length, Sepal.Width, color = Species)) +  
  geom_point() +  
  labs(title = "Sépales des iris", x = "Longueur (cm)", y = "Largeur (cm)",  
        color = "Espèce", caption = "Données : Fisher (1936)") + # un titre par aesthetic  
  theme_bw() +  
  theme(legend.position = "bottom", # "none" : pas de légende  
        axis.text.x = element_text(angle = 90, hjust = 1)) # étiquettes verticales  
  
ggsave("figures/iris.png", width = 16, height = 10, units = "cm", dpi = 300) # dernier graphique  
ggsave("figures/iris.pdf", plot = p, width = 16, height = 10, units = "cm")
```

- **labs()** : un titre pour chaque *aesthetic* utilisé (**x**, **y**, **color**, **fill**...); **toujours** les unités
- **ggsave()** : l'extension choisit le format; **width**, **height** et **dpi** fixent la **taille réelle** dans l'article
- Un graphique **ggplot2** est un **objet**: **p <- ggplot(...)**, puis **p + theme_bw()**, **ggsave(plot = p)**

👉 À vous : *limits* ou zoom ?

Les tailles de 50 cellules, dont 3 très grosses ($> 200 \mu\text{m}$). On écrit :

```
ggplot(cellules, aes(x = ligne, y = taille)) +  
  geom_boxplot() +  
  ylim(0, 100)
```

1. Que deviennent les 3 grosses cellules ?
 2. La médiane affichée est-elle celle des 50 cellules ?
 3. Comment voir le détail sous $100 \mu\text{m}$ sans changer les boxplots ?
-
1. Elles sont **retirées des données** (`ylim() = scale_y_continuous(limits = )`), avec un simple *warning* **Removed 3 rows**
 2. Non : médiane, quartiles et moustaches sont calculés sur les **47** restantes
 3. **`coord_cartesian(ylim = c(0, 100))`** : les 3 points sortent du cadre, mais comptent dans le calcul

| 5

PRÉPARER LES DONNÉES

ggplot2 attend le format long

Un boxplot des notes par type d'évaluation : quelle colonne mettre en **x** ?

grades.tsv : une colonne par évaluation

ID_etu	D1	D2	E1
Etu_1	90	87.5	94.44
Etu_2	86.67	88	83.33
Etu_3	83.33	70	56.67

Aucune : le type d'évaluation est **dans les noms de colonnes**. Il faut une colonne **Type** et une colonne **Note**.

ggplot2 attend le format long

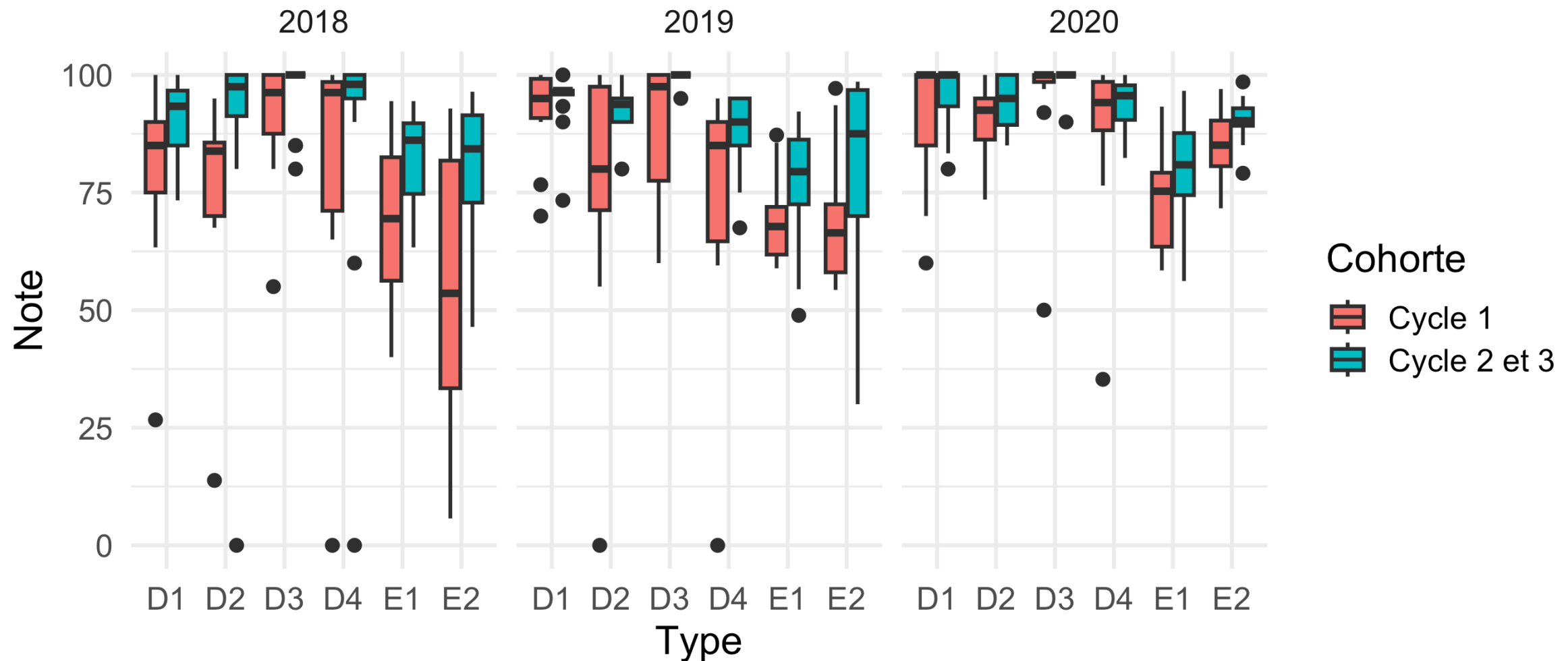
une ligne par étudiant x évaluation

ID_etu	Type	Note
Etu_1	D1	90
Etu_1	D2	87.5
Etu_1	E1	94.44
Etu_2	D1	86.67
Etu_2	D2	88
Etu_2	E1	83.33
Etu_3	D1	83.33
Etu_3	D2	70
Etu_3	E1	56.67

```
notes_long <- notes |> pivot_longer(D1:E2, names_to = "Type", values_to = "Note")
ggplot(notes_long, aes(x = Type, y = Note)) + geom_boxplot()
```


Le format long, puis le graphique

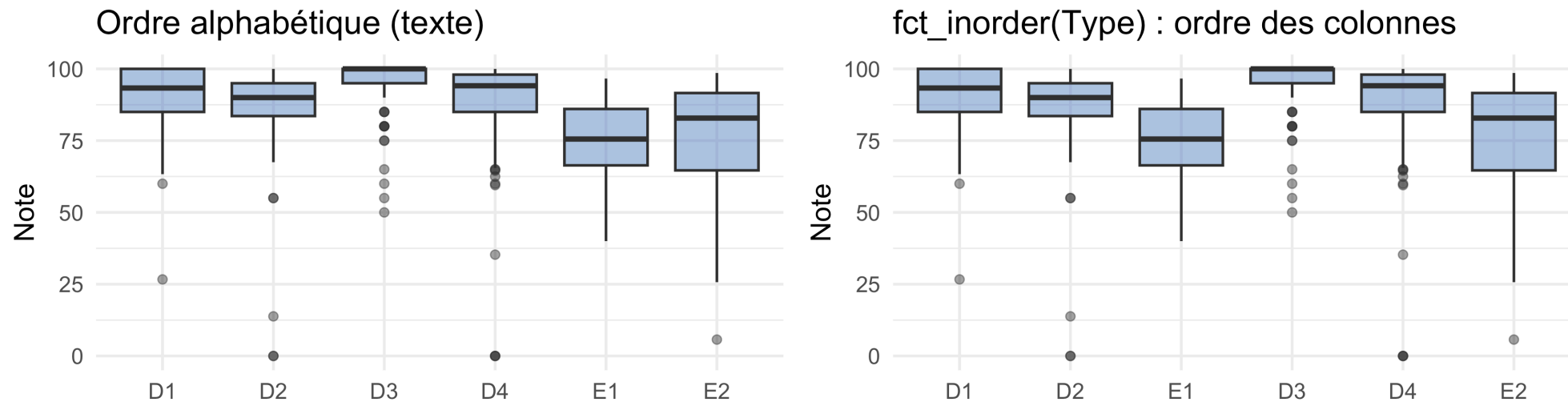
```
notes_long <- notes |> pivot_longer(D1:E2, names_to = "Type", values_to = "Note")
ggplot(notes_long, aes(x = Type, y = Note, fill = Cohorte)) +
  geom_boxplot() +
  facet_wrap(~ Annee)
```



Une colonne par *aesthetic* : **Type** en x, **Note** en y, **Cohorte** en couleur, **Annee** en facettes.

L'ordre des catégories

Les évaluations ont eu lieu dans l'ordre D1, D2, E1, D3, D4, E2. Sur le graphique, elles sont dans l'ordre **alphabétique** :



Fonction (**forcats**)

```
factor(x, levels = c("D1", "D2",  
"E1"))
```

```
fct_inorder(x)
```

```
fct_infreq(x)
```

```
fct_reorder(x, y)
```

Ordre obtenu

celui qu'on écrit

ordre d'apparition dans les données

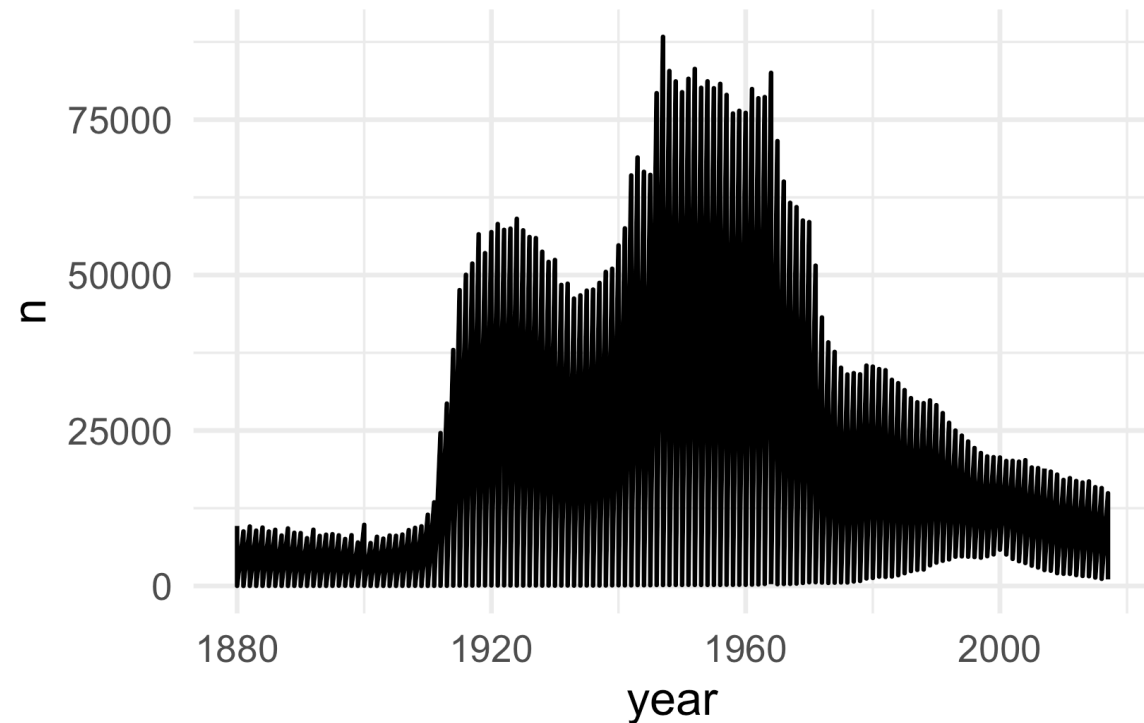
du plus fréquent au moins fréquent
(**geom_bar()** trié)

selon la médiane de **y** (boxplots ou barres
triés)

Les lignes : l'*aesthetic* **group**

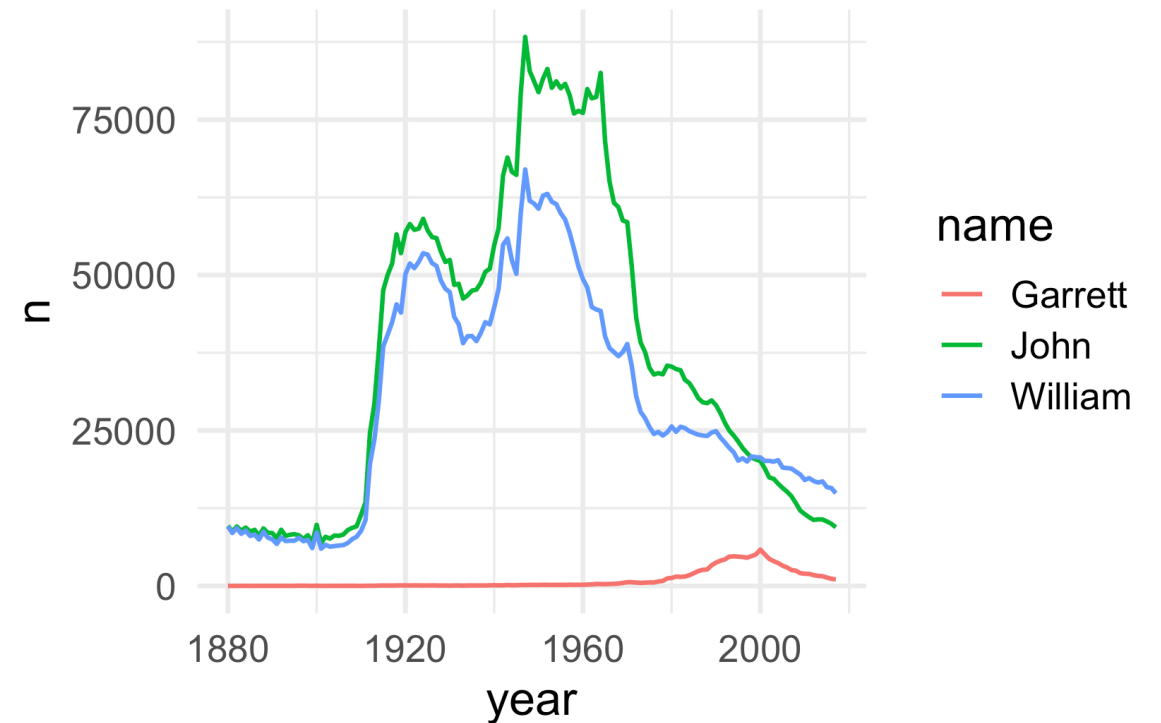
L'évolution de 3 prénoms de garçons (**babynames**) :

```
trois |>  
  ggplot(aes(year, n)) + geom_line()
```



Une **seule** ligne relie toutes les lignes du tableau dans l'ordre des x : elle zigzague d'un prénom à l'autre chaque année.

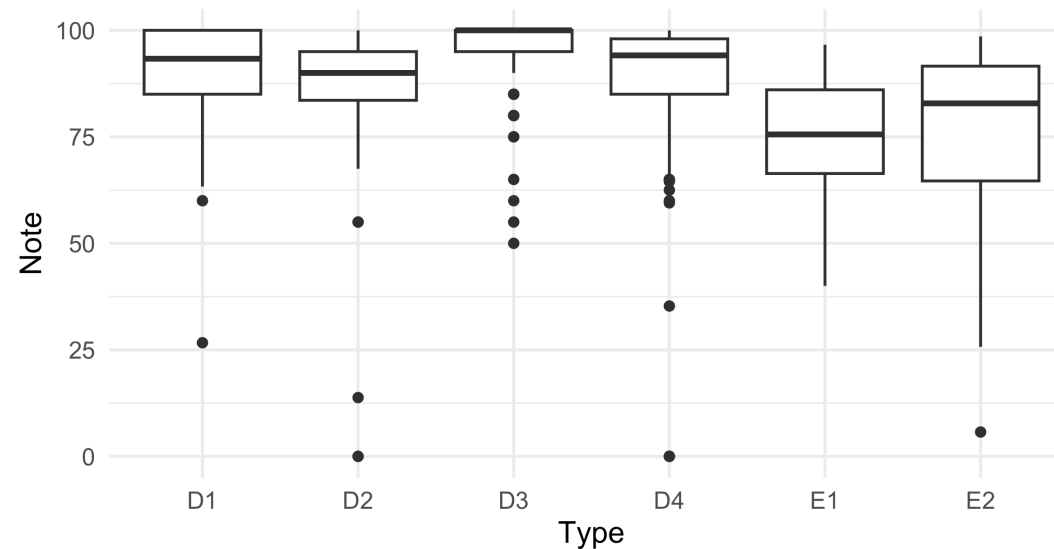
```
trois |>  
  ggplot(aes(year, n, color = name)) + geom_line()
```



color (ou **group = name**) découpe les données : **une ligne par prénom**.

Les **NA** dans un graphique

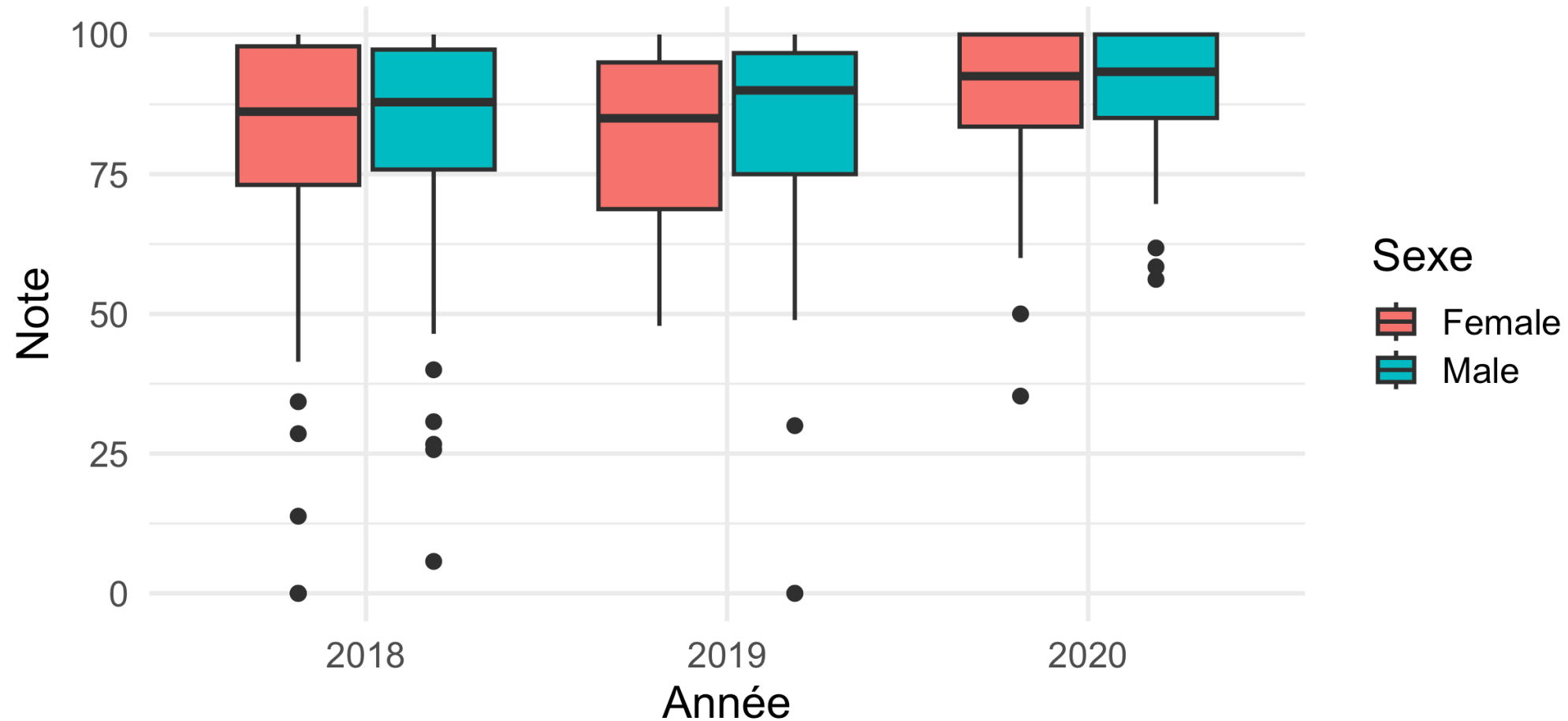
```
ggplot(notes_long, aes(Type, Note)) + geom_boxplot()  
#> Warning: Removed 1 row containing non-finite outside the scale range  
#> (`stat_boxplot()`).
```



- Une note **D2** manque : **ggplot2** la **retire** et prévient par un *warning*, sans s'arrêter
- Dans un rapport Quarto, les *warnings* sont souvent masqués : comptez les **NA** avant de tracer
- Dans une colonne **catégorielle**, **NA** devient une catégorie à part, grise, dans la légende : souvent le signe d'une jointure ratée ou d'une valeur mal saisie

Même logique pour les catégories mal saisies (module 5) : "**Male**" et "**male** " font **deux** couleurs dans la légende. Un graphique est souvent le moyen le plus rapide de les repérer.

👋 À vous : quel code a produit ce graphique ?



Écrivez le code, à partir de **notes** et **sexes** (une ligne par étudiant : **ID_etu**, **Sex**).

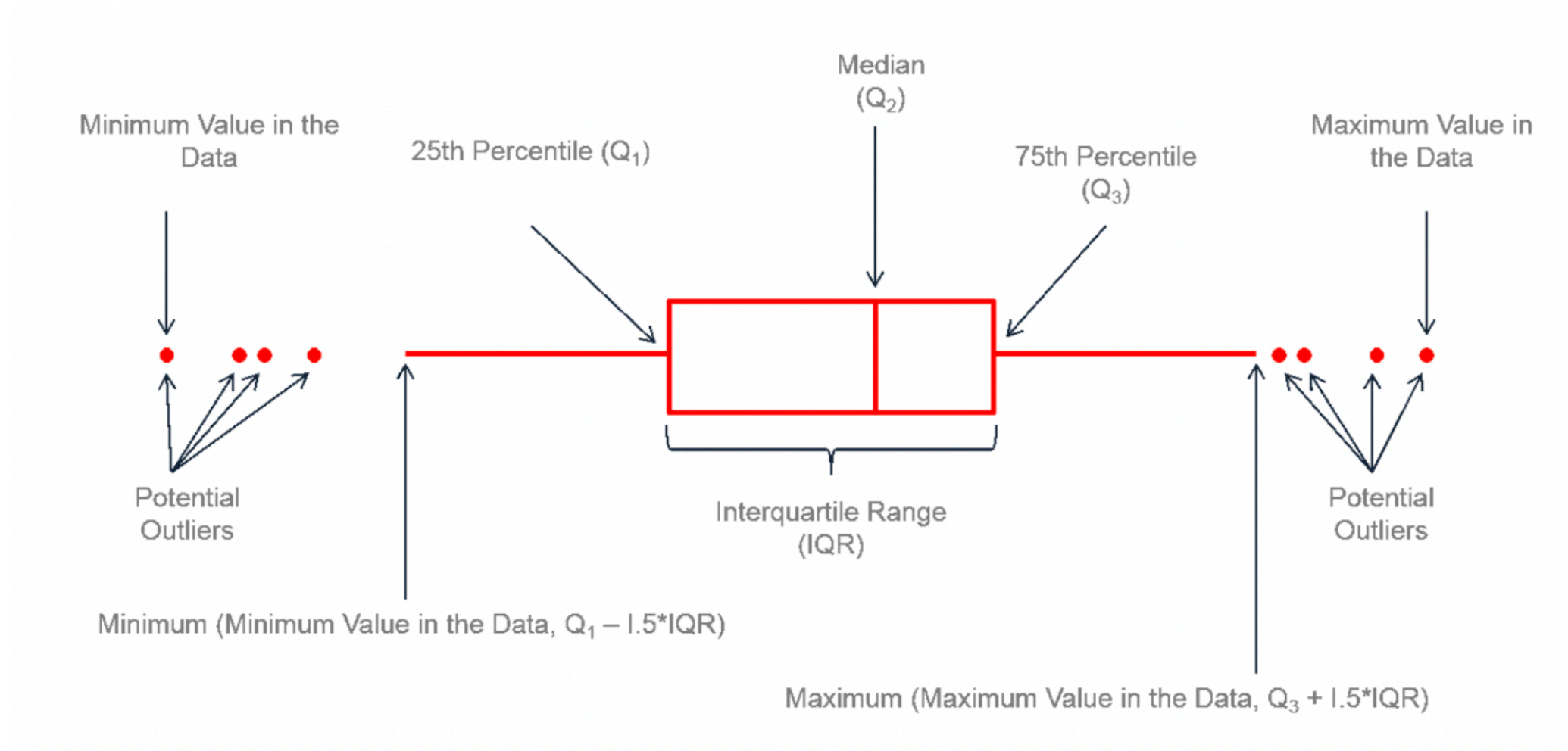
```
notes |>
  pivot_longer(D1:E2, names_to = "Type", values_to = "Note") |> # une ligne par note
  left_join(sexes, by = "ID_etu") |> # ajouter le sexe
  ggplot(aes(x = factor(Annee), y = Note, fill = Sex)) + # factor() : 3 groupes
  geom_boxplot() +
  labs(x = "Année", fill = "Sexe")
```

TP2

| 6

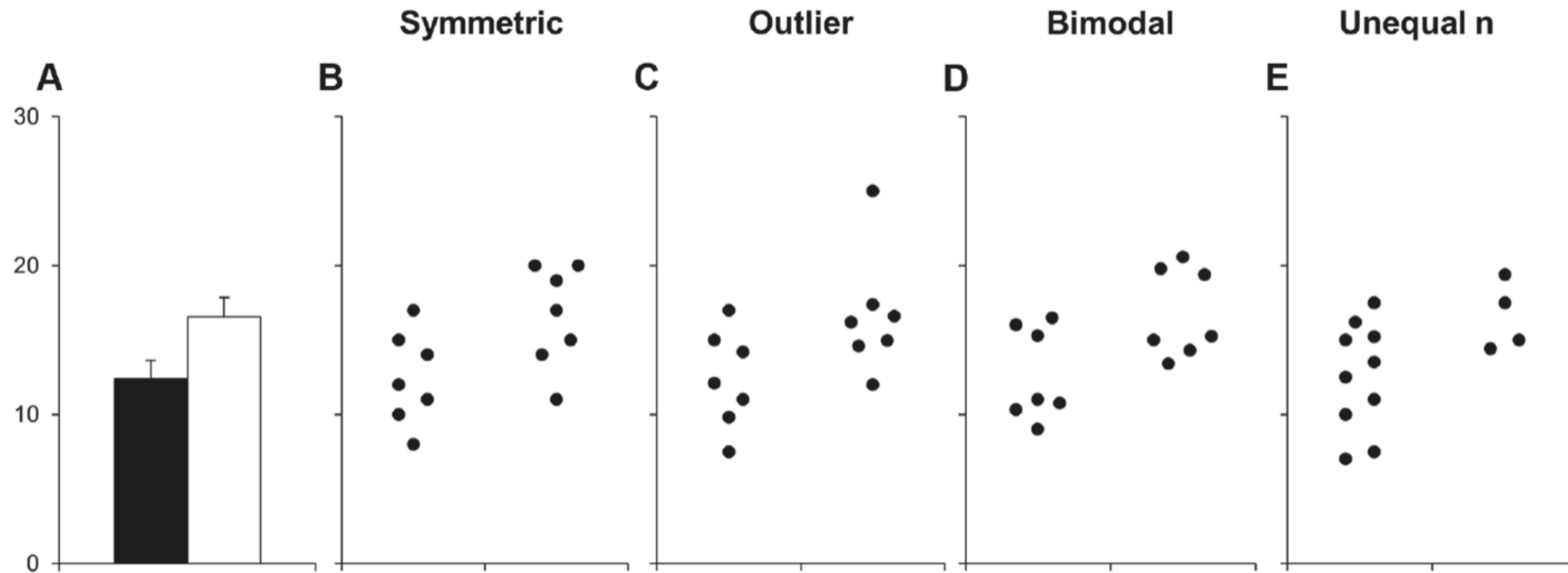
GRAPHIQUES DE LA BIOINFORMATIQUE

Le boxplot



- La boîte : **50 %** des valeurs (du 1^{er} au 3^e quartile) ; le trait : la **médiane**
- Les moustaches : jusqu'à $1.5 \times \text{IQR}$; au-delà, les points atypiques
- Il ne montre **pas l'effectif** : 4 points et 400 points donnent la même boîte. Ajoutez les points (`geom_jitter()`)

Les limites du barplot

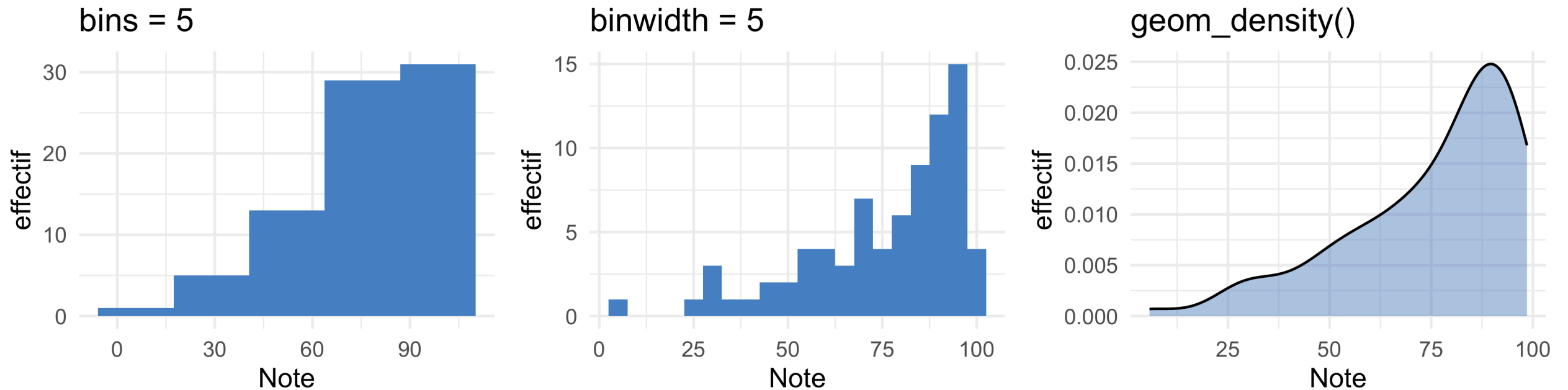


Les cinq jeux de données B à E donnent **le même barplot** A (moyenne $\pm$ erreur type) : symétrique, valeur aberrante, bimodal, effectifs inégaux.

Weissgerber et al. (2015). *Beyond bar and line graphs: time for a new data presentation paradigm*. PLoS Biology 13(4): e1002128.

Un barplot convient aux **comptes** et aux **totaux**. Pour une variable continue mesurée sur quelques réplicats, montrez **les points**.

L'histogramme : la largeur des classes



- Les notes de E2 : avec 5 classes, on ne voit pas le groupe d'échecs ; avec des classes de 5 points, il apparaît
- `ggplot2` choisit 30 classes par défaut et le dit : ``stat_bin()` using `bins = 30``.
Pick better value : choisissez `bins` ou `binwidth`
- La densité lisse l'histogramme ; elle peut déborder hors des valeurs possibles (au-delà de 100)

Les résultats d'une expression différentielle

```
#>      gene  baseMean log2FoldChange      pvalue      padj
#> 1 TSPAN6   585.09000      0.13504 4.3732e-01 7.9465e-01
#> 2  TNMD     0.33438     -1.78760 6.5755e-01      NA
#> 3  ICAM1    79.83000      2.87280 2.5421e-18 5.5596e-15
#> 4 CRABP2 1938.70000     -1.41680 6.2468e-27 8.1970e-23
```

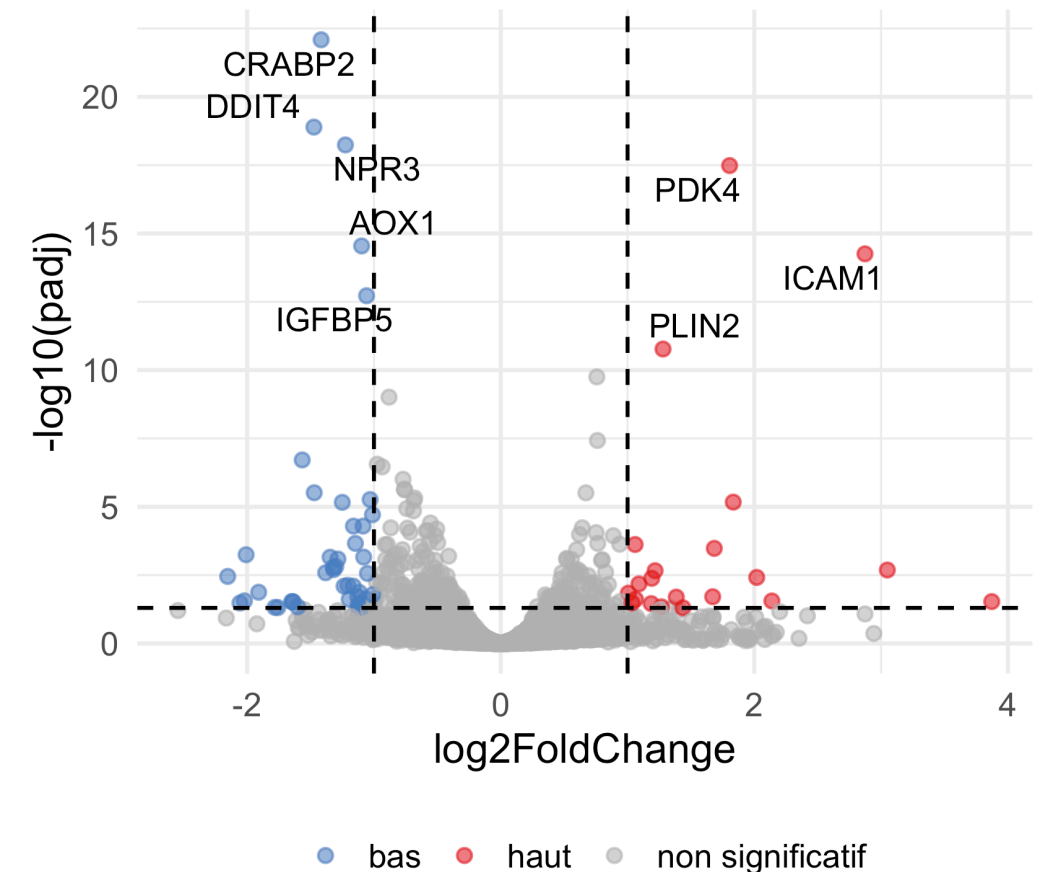
[/home/public/visu/de_resultats.csv](#) : deux conditions, 3 réplicats chacune, 19 320 gènes. Produit par DESeq2 (module 9).

- **log2FoldChange** : l'ampleur du changement. +1 = deux fois plus exprimé, -1 = deux fois moins
- **pvalue, padj** : la **significativité** ; **padj** est corrigée pour les tests multiples (module 9)
- **padj** vaut **NA** pour les gènes **non testés** (trop peu exprimés) : ici, 6308 gènes

Le volcano plot

```
de_v <- de |>
  filter(!is.na(padj)) |>
  mutate(statut = case_when(
    padj < 0.05 & log2FoldChange > 1 ~ "haut",
    padj < 0.05 & log2FoldChange < -1 ~ "bas",
    .default = "non significatif"))

ggplot(de_v, aes(x = log2FoldChange,
                  y = -log10(padj), color = statut)) +
  geom_point(alpha = 0.6) +
  geom_vline(xintercept = c(-1, 1), linetype = 2) +
  geom_hline(yintercept = -log10(0.05),
             linetype = 2) +
  geom_text_repel(data = top, aes(label = gene)) +
  scale_color_manual(values = c(
    haut = "red", bas = "blue",
    `non significatif` = "grey70"))
```



Lire un volcano plot

- x : $\log_2 \text{fold change}$. À droite : plus exprimé ; à gauche : moins exprimé
- y : $-\log_{10}(\text{padj})$. Plus c'est haut, plus c'est significatif : $\text{padj} = 0.05 \rightarrow 1.3$; $0.001 \rightarrow 3$; $10^{-20} \rightarrow 20$
- En haut à droite et en haut à gauche : les gènes à la fois très changés et significatifs
- Les pointillés : des seuils choisis par l'analyste (ici $|\log_2\text{FC}| > 1$ et $\text{padj} < 0.05$). Les changer change la liste des gènes

Deux couches de données :

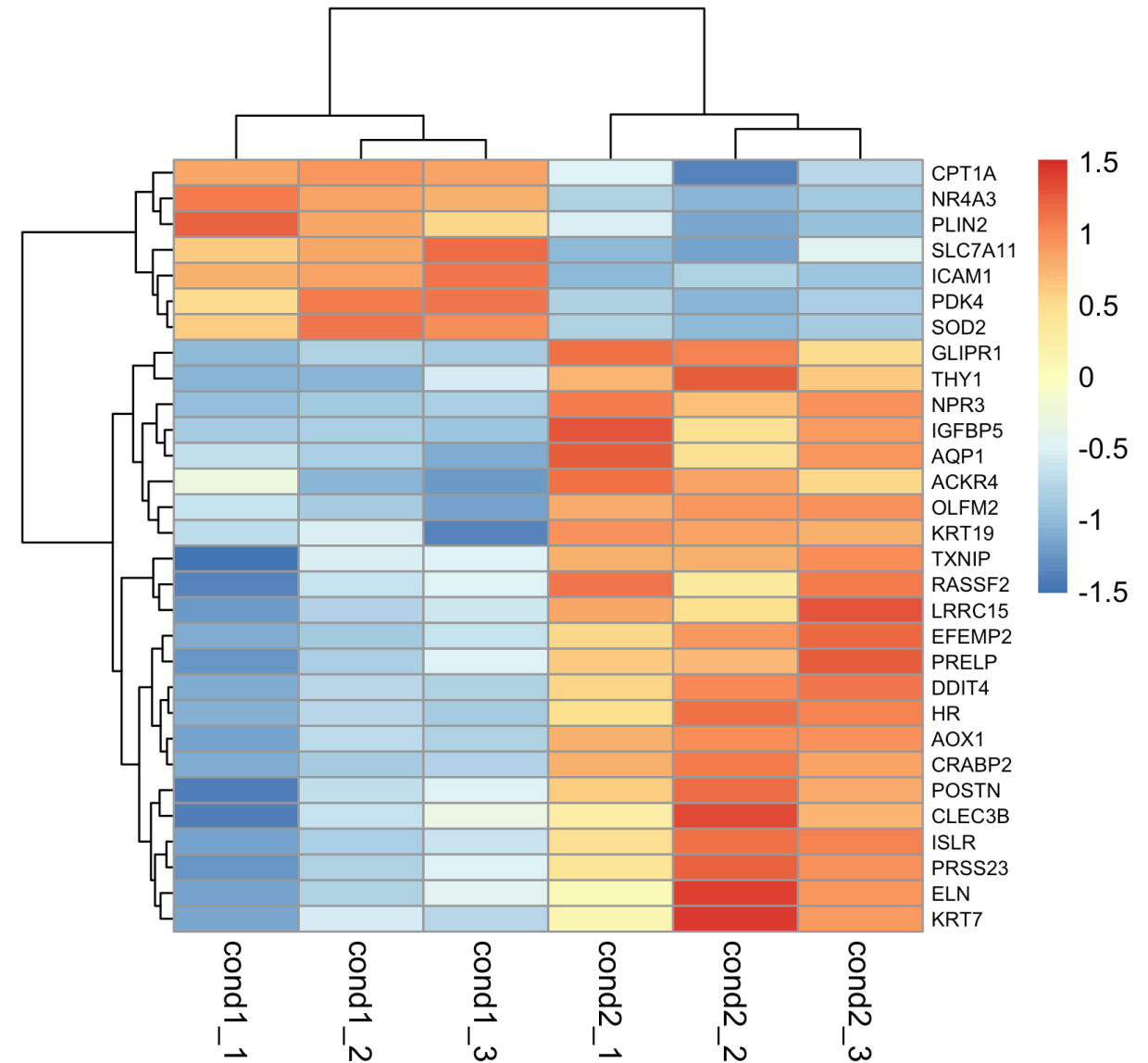
`geom_text_repel(data = top, ...)` n'utilise que les 8 gènes de `top`, sur le même graphique. `ggrepel` écarte les étiquettes pour qu'elles ne se chevauchent pas.

« haut » : plus exprimé dans **quelle** condition ?
Cela dépend du **niveau de référence** de la comparaison. Vérifiez sur un gène (TP3) ou lisez l'en-tête des résultats (module 9) avant d'écrire « induit ».

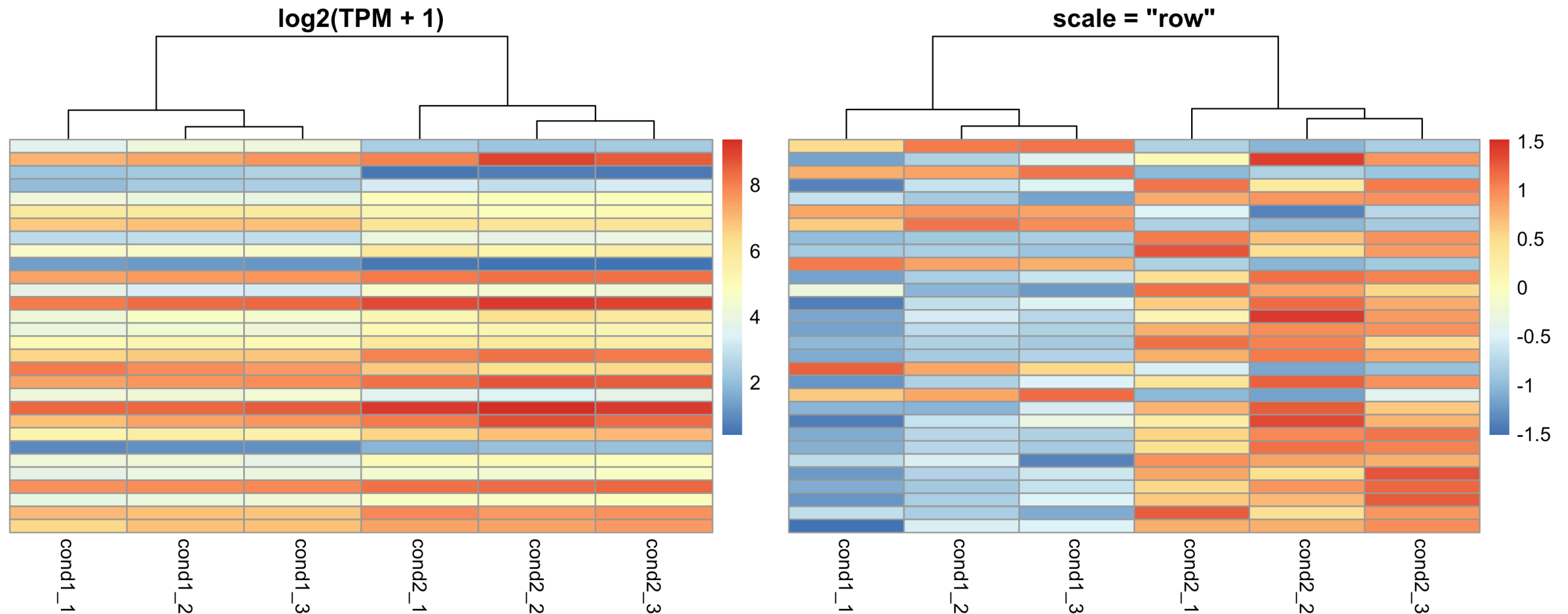
La heatmap

```
library(pheatmap)
genes <- de |> slice_min(padj, n = 30) |>
  pull(gene)
m <- tpm |>
  filter(gene %in% genes) |>
  column_to_rownames("gene") |>
  as.matrix()
pheatmap(log2(m + 1), scale = "row")
```

- Une **matrice** : un gène par ligne, un échantillon par colonne
- **$\log_2(m + 1)$** : les TPM vont de 0 à 20 000 ; le **+ 1** évite **$\log_2(0) = -\text{Inf}$**
- **`scale = "row"`** : chaque gène centré-réduit. On compare les **échantillons** entre eux, gène par gène
- Les arbres : un **regroupement** des gènes et des échantillons qui se ressemblent

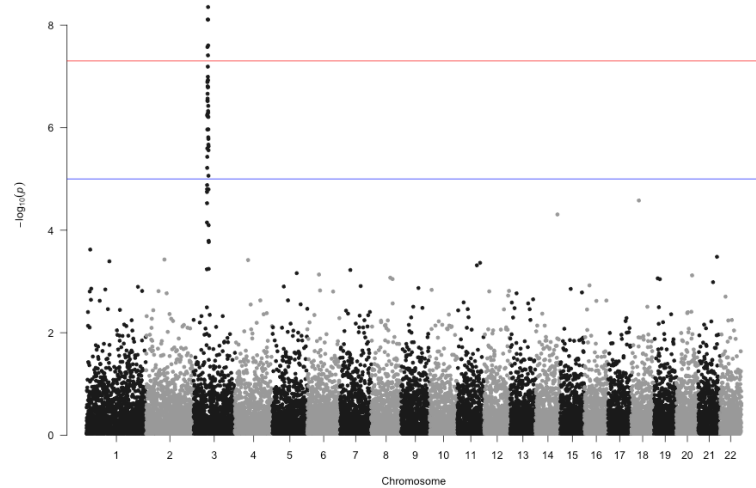


Sans **scale = "row"**

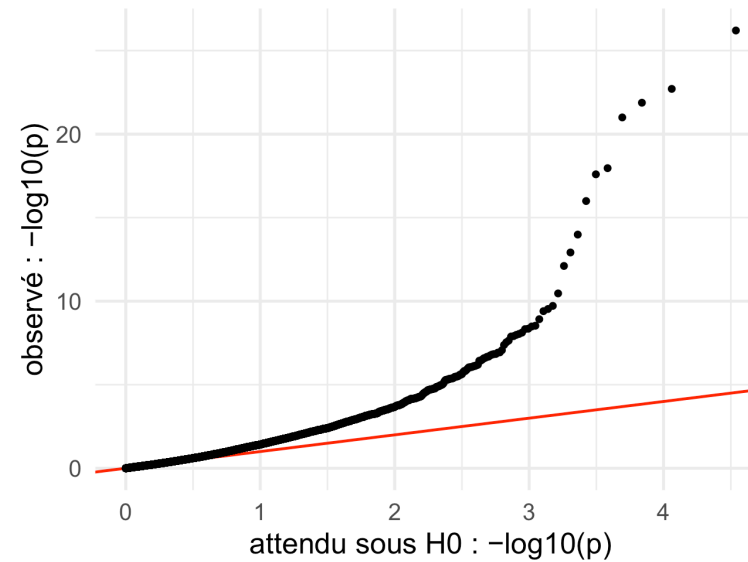


- À gauche : la couleur montre surtout **quels gènes sont très exprimés**. Les différences entre conditions sont invisibles
- À droite : chaque gène est comparé à **sa propre moyenne** : les deux conditions se séparent
- Une heatmap sans légende des couleurs, ou sans dire si les lignes sont centrées, ne se lit pas

D'autres graphiques de la bioinformatique



Manhattan plot : $-\log_{10}(p)$ de chaque SNP le long du génome (études d'association, GWAS)



Q-Q plot des p-values : sous H_0 , les points suivent la diagonale ; les gènes qui s'en écartent en haut sont les vrais signaux

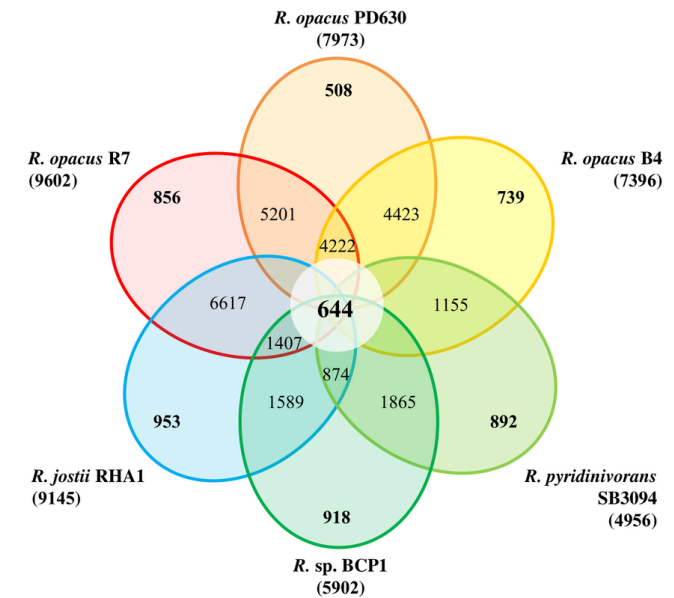


Diagramme de Venn (ou UpSet) : les gènes communs à plusieurs listes. Réseaux : module 10

BONNES PRATIQUES

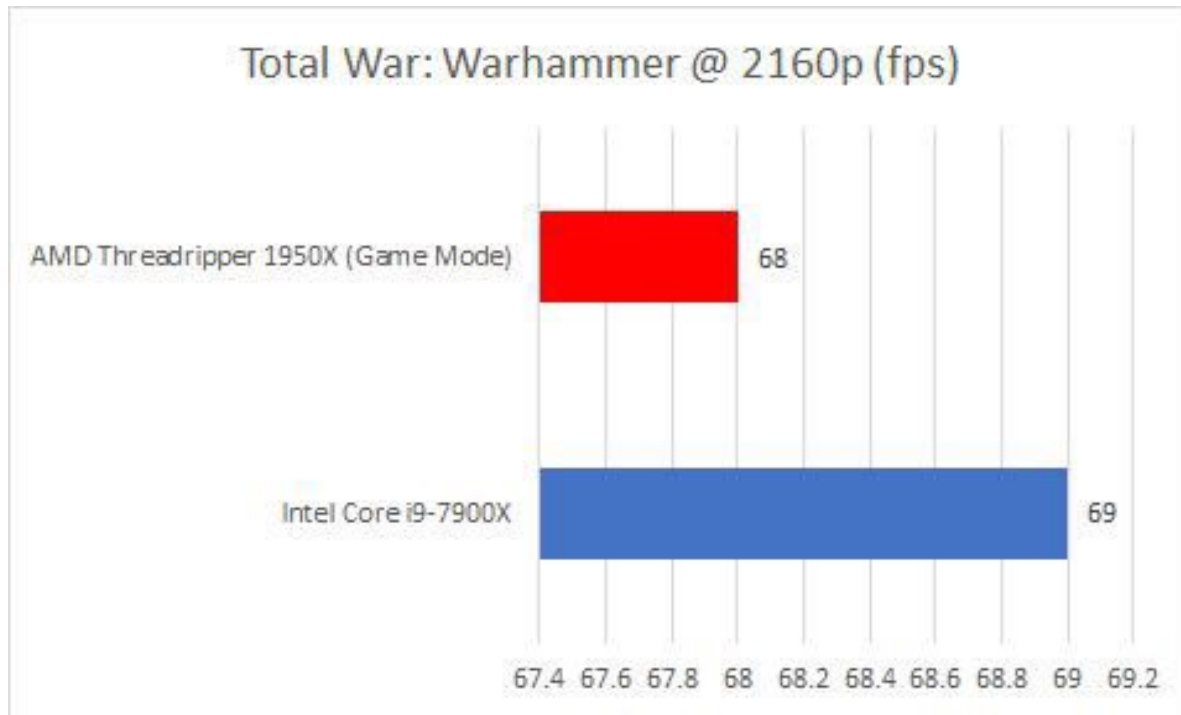
Choisir le graphique

1. **Quelle est la question ?** Comparer des groupes, montrer une distribution, une relation, une évolution
2. **Choisissez le visuel** pour la question et le type de variables, pas pour l'effet
3. **Montrez les données** quand il y en a peu : des points plutôt qu'une barre
4. Donnez le **contexte** : titre, axes avec unités, légende, effectifs, source
5. **Simplifiez** : pas de 3D, pas d'ombres, pas de couleurs qui ne codent rien

Les erreurs fréquentes

- Axes sans titre ou sans unités
- **Axe tronqué** : un barplot qui ne commence pas à 0
- Des parts qui ne font pas 100 % (*pie chart*)
- Un mauvais type de graphique pour les données
- *Cherry-picking* : ne montrer que les données qui arrangent
- Pas de titre, pas de légende
- **Couleurs** : rouge et vert ensemble ($\approx 8\%$ des hommes ne les distinguent pas) ; préférer **viridis** ou rouge/bleu
- Une **3D** inutile, qui déforme les hauteurs
- Plusieurs variables sans rapport sur un même graphique (deux axes y)
- **Échelles libres** entre facettes sans le signaler

👉 À vous : qu'est-ce qui ne va pas ?



- L'axe des x commence à **67.4**, pas à 0 : 68 contre 69 images par seconde, une différence de 1.5 %, devient une barre **2.7 fois** plus longue
- Pour un barplot, la **longueur** de la barre est le message : l'axe doit partir de 0
- Pour des points ou des lignes, un axe qui ne part pas de 0 est acceptable, s'il est clairement gradué

https://getdolphins.com/wp-content/uploads/2018/02/DG5UK_WUAAAVQe5.jpg

👉 À vous : qu'est-ce qui ne va pas ?

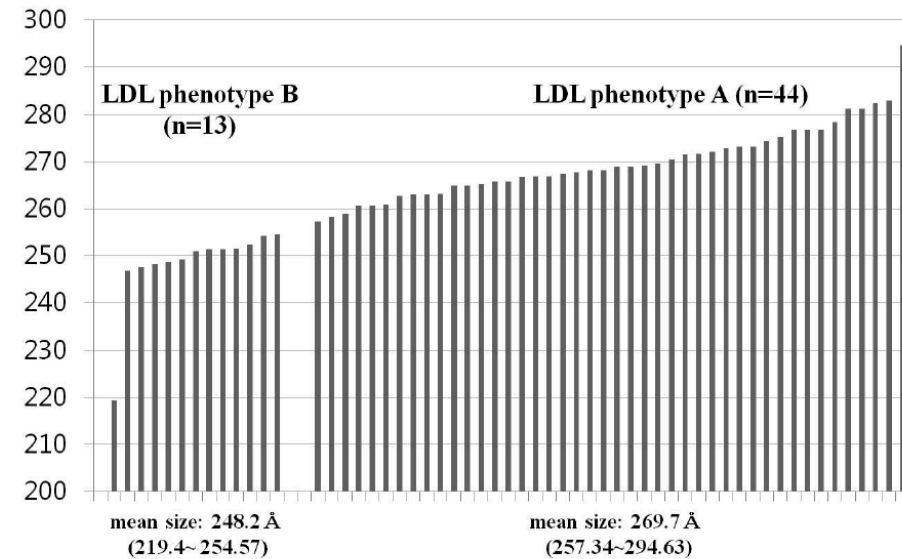
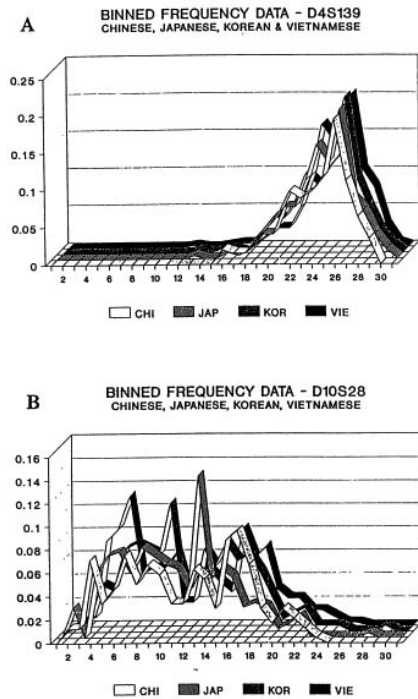
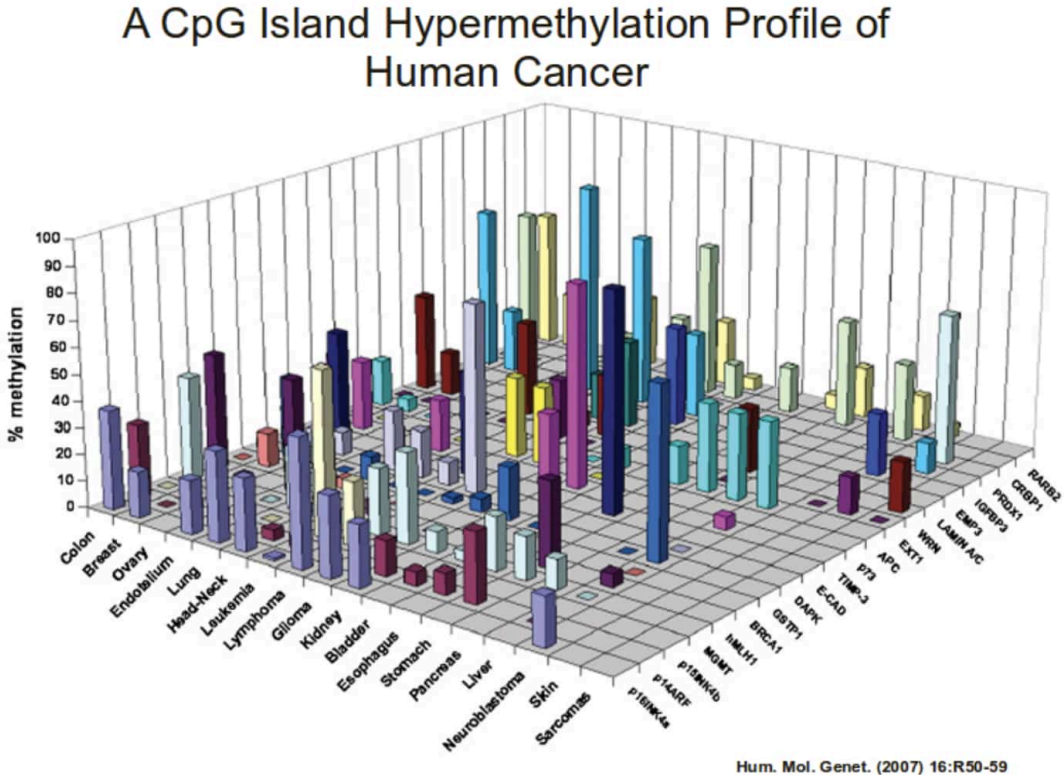


Fig. 1. Distribution of low-density lipoprotein (LDL) particle size in all study subjects (LDL phenotypes A and B). *LDL phenotype A group* (mean size: 269.7 Å, n = 44), subjects with buoyant-mode profiles [peak LDL particle diameter ≥ 264 Å] including intermediate LDL subclass pattern [$256 \text{ Å} \leq \text{peak LDL particle diameter} \leq 263 \text{ Å}$]; *LDL phenotype B group* (mean size: 248.2 Å, n = 13), subjects with dense-mode profiles [peak LDL particle diameter $\leq 255 \text{ Å}$]

- À gauche : la **3D** n'ajoute aucune information ; les courbes se cachent les unes les autres
- À droite : une barre **par individu**, triée, sur un axe qui part de 200 : difficile de comparer les deux groupes. Des points par groupe (**geom_jitter()** + boxplot) diraient la même chose en un coup d'œil



- À gauche : six variables sur un même visuel, avec **deux axes y** : on ne sait plus quelle barre ou quelle ligne lire sur quel axe
- À droite : un tableau tissu $\times$ gène en **barres 3D** : des barres en cachent d'autres. Une **heatmap** (couleur = % de méthylation) montrerait tout

TP3

| 7

ACTIVITÉ : AUDITER UN SCRIPT

Auditer un script **ggplot2** généré par IA

« À partir de `grades.tsv`, compare les deux cohortes : (1) la distribution des notes de chaque évaluation, dans l'ordre du trimestre (D1, D2, E1, D3, D4, E2) ; (2) un barplot de la note moyenne de chaque cohorte. Les notes sont sur 100. »

```
library(tidyverse)
# Script vérifié : s'exécute sans erreur.
notes <- read_tsv("/home/public/tidyverse/grades.tsv")

notes_long <- notes |>
  pivot_longer(D1:E2, names_to = "Type", values_to = "Note")

# 1. Distribution des notes par évaluation et par cohorte
ggplot(notes_long, aes(x = Type, y = Note, fill = Cohorte)) +
  geom_boxplot() +
  scale_y_continuous(limits = c(50, 100)) + # les notes sont sur 100
  theme_bw()

# 2. Note moyenne par cohorte
ggplot(notes_long, aes(x = Cohorte, y = Note, fill = "steelblue")) +
  geom_col() +
  labs(y = "Note moyenne")
```

`grades.tsv` (premières lignes)

ID_etu	D1	E2	Annee	Cohorte
Etu_1	90.00	92.86	2018	Cycle 1
Etu_2	86.67	90.00	2018	Cycle 1
Etu_3	83.33	30.71	2018	Cycle 1
Etu_4	83.33	82.86	2018	Cycle 1

Effectifs

79 étudiants, 474 notes, 1 NA

Auditer : la démarche

Le script s'exécute sans erreur. Au moins trois de ses graphiques ou de ses détails sont pourtant faux.

1. **Lire sans exécuter** : que fait réellement chaque ligne ? Lesquelles sont suspectes ?
2. **Vérifier dans RStudio** : lire **tous** les messages, les **axes** et les **légendes**, et refaire les calculs avec `summarise()`
3. **Corriger** et commenter chaque correction

Indices : relisez la demande (l'**ordre** des évaluations, ce qu'est une **moyenne**), lisez les *warnings*, et regardez les valeurs sur l'axe y.
Certaines lignes sont correctes : ne corrigez pas ce qui fonctionne.

Ressources

- *R for Data Science* (2^e éd., H. Wickham, M. Çetinkaya-Rundel, G. Grolemund) : <https://r4ds.hadley.nz>, chapitres 1, 9, 10 et 11 (visualisation)
- *ggplot2: Elegant Graphics for Data Analysis* (3^e éd., H. Wickham) : <https://ggplot2-book.org>
- *R Graphics Cookbook* (W. Chang) : <https://r-graphics.org>, une recette par graphique
- *Fundamentals of Data Visualization* (C. Wilke) : <https://clauswilke.com/dataviz/>, choisir et soigner un graphique
- Aide-mémoire `ggplot2` : <https://rstudio.github.io/cheatsheets/data-visualization.pdf>
- Galerie : <https://r-graph-gallery.com>, des centaines de graphiques avec leur code

Données : `iris` (Fisher, 1936), `diamonds` (package `ggplot2`), `babynames` (Social Security Administration, États-Unis ; package `babynames`, H. Wickham), notes anonymisées du cours (`grades.tsv`), résultats DESeq2 d'une expérience RNA-seq (`de_resultats.csv`, `tpm.csv`).